

| Practical Messaging — Day One

A 101 guide to messaging. Ian Cooper. (X, BlueSky and Hachyderm: ICooper)

The Process Boundary

What sending messages between processes commits you to.
Two slides, and then straight into coupling.

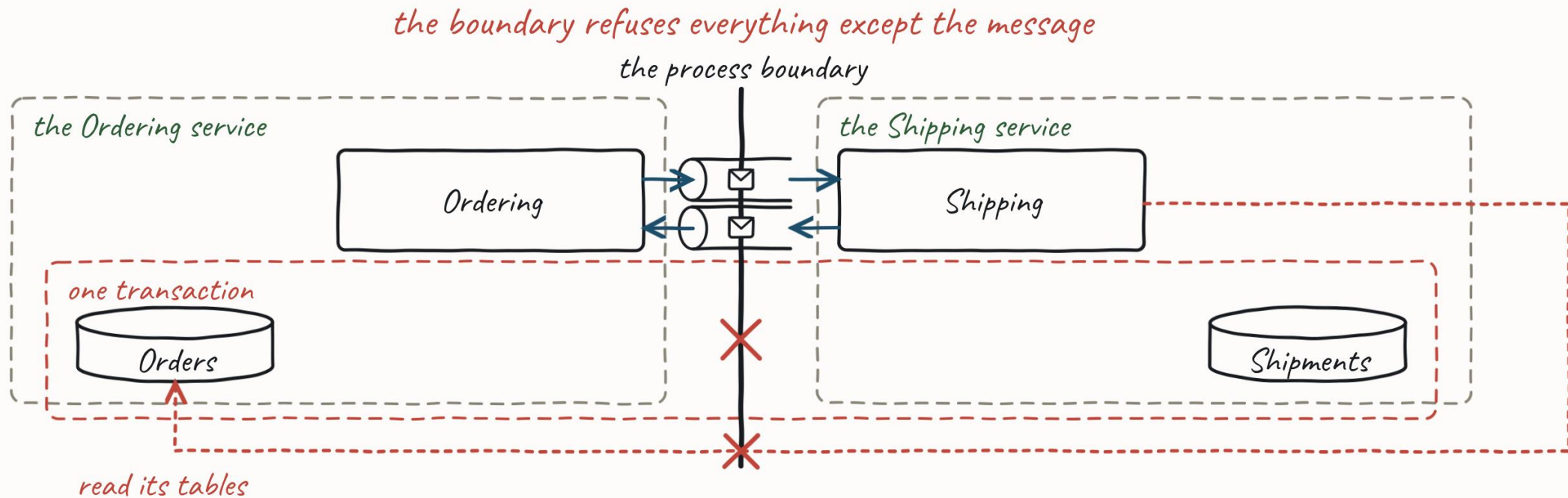
Messages In, Private Data, No Shared Transaction

We are going to spend two days sending messages between processes. Before anything else, **what does drawing that boundary decide for you?**

- **The only way to get work done in another process is to send it a message.** Each service has its own accepted message types, and its own data requirements for partners submitting work.
- **The data behind the boundary is private.** A request does not describe the shape of the data inside; it describes the work. Nobody reads your tables.
- **No transaction spans two services.** Transactions — 2PC included — happen *within* a boundary, never across one. You do not exchange transactions with your business partners, because their mistake would lock your database. Without transactions you communicate through **multiple messages over time**.

Messages In, Private Data, No Shared Transaction

No transaction spans two services. Consistency stops being something you declare and becomes something you design.



Robust — Guaranteed Delivery

The boundary costs you transactions. It buys you this — and this one is cheap.

- We want the work **not to be lost** when something we depend on is slow, overwhelmed, or down.
- **Store and forward.** The work waits somewhere durable until whoever does it is ready. **The outage becomes a delay.**
- **You do not need to reorganise anything to get this.** A single team with a single web application can have it on Monday.

| One team, one service, one queue. Robustness without reorganising the company.

Coupling

What the process boundary bought you, and what you can still give back.

What the Process Boundary Already Bought You

The classic coupling scale runs Content → Common → Control → Stamp → Data, tightest first. **Drawing a process boundary takes the top two off the table.**

- **Content** — one party reaches into the other's internals. Across a network this can only survive as a degenerate form: depending on undocumented behaviour, a private endpoint, someone else's table. There is no pointer into another process's memory.
- **Common** — both parties share the same mutable store. Uncontrolled propagation of change; nobody owns the schema. Separate processes with **private data** do not have one.

This is what *Messages In, Private Data, No Shared Transaction* buys you: the two tightest forms of coupling are not available across a process boundary.

What the Process Boundary Already Bought You

the two worst kinds of coupling are the two you cannot have any more – that is what you bought

tightest
prevented ~~*Content*~~ *one party reaches into the other's internals*

prevented ~~*Common*~~ *both parties share the same mutable store*

the process boundary
separate processes, private data,
no shared transaction

Control *one party tells the other what to do*

Stamp *a whole structure is passed, and part of it used*

Data *exactly what is needed is passed, and nothing else*
loosest

What's Left Is in the Message

You cannot avoid the other three. Every message you send picks one, whether or not you meant to.

- **Control** — you tell the other party *what to do* rather than *what happened*. A what-to-do flag in the payload. The receiver's behaviour is now your business.
- **Stamp** — you share a composite structure and each of you uses only part of it. A change to a field you never read can still break you.
- **Data** — you share only the elementary values you each actually need.

| Stamp coupling is the one you will meet every day.

| The boundary chose the first two for you. The message is where you choose the third.

Must We Both Be Up?

The second axis is **time**. If both parties must be present for the communication to succeed, they are **temporally coupled**: the availability of one becomes the availability of the other.

	Synchronous conversation	Asynchronous conversation
Shape	Request → Reply	Store and forward
Both present?	Yes	No — pick it up later
Analogy	A phone call	Snail mail
Temporal coupling	Introduces it	Avoids it

- **It is the axis that multiplies your outages.** Call a service and wait, and your availabilities multiply: four services at 99.9% leaves you at **99.6%**, before anything has actually failed.
- **Send a message and don't wait, and they don't.** Guaranteed delivery means the message outlives their outage and is processed when they come back.

| If my availability depends on yours, I have bought your outages.

Two Axes, Not One Scale

"Loose coupling" is not a single dial. What we are coupled *about* and whether we must both be *up* move independently:

- **gRPC with a flat DTO** — data coupled, but temporally coupled.
- **A command message with a what-to-do flag** — control coupled, but temporally decoupled.
- **A shared database** — common coupled, and temporally decoupled.
- **An event carrying a whole entity** — stamp coupled, temporally decoupled.

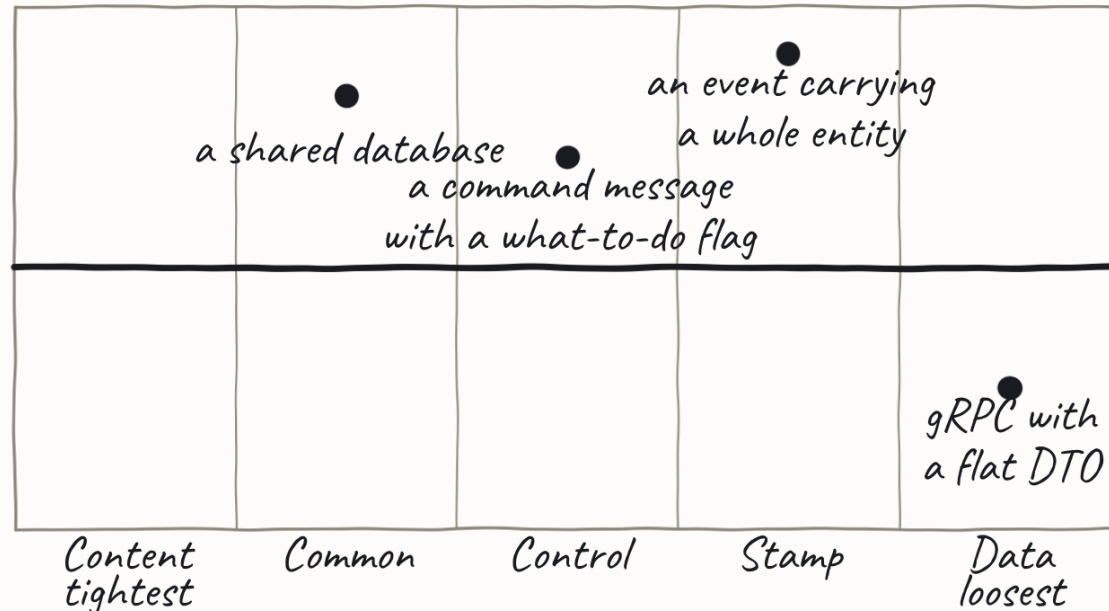
Two Axes, Not One Scale

"loosely coupled" is a question with two answers

must we both be up?

*no –
both can be down*

*yes –
both must be up*



what are we coupled about?

Integration Styles

Four ways to communicate between processes — and what each one gives back.

File Transfer

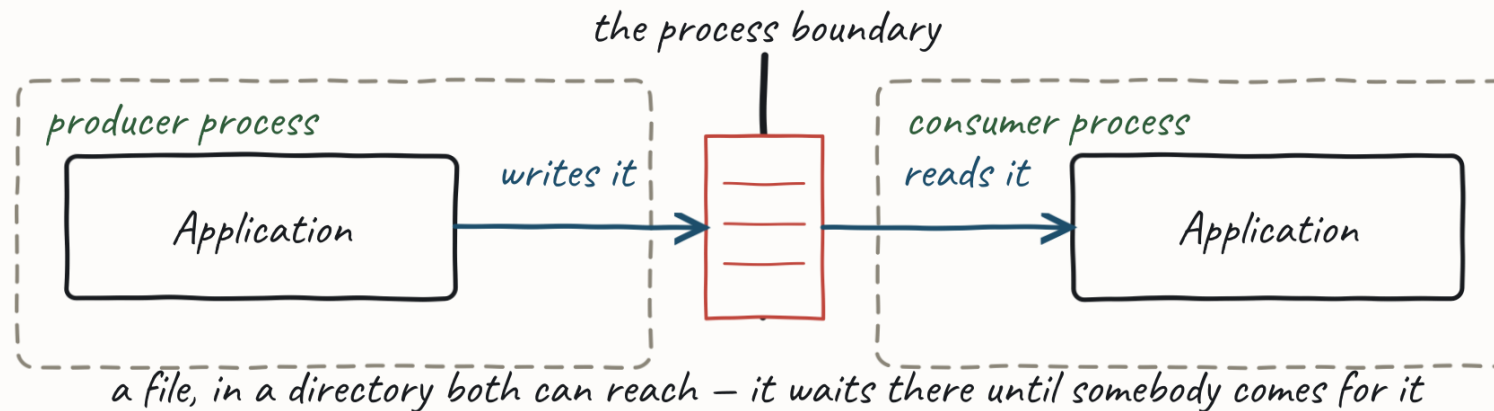
One application writes a file; another reads it later. **The file is the contract.**

- **Asynchronous conversation** — nobody has to be up at the same time.
- **Data coupling** — if you keep the file format elementary.

And the file is a message. A batch of them, in a channel that happens to be a filesystem.

File Transfer

the file is the contract – its format and where it lives, and nothing else is agreed



and every one of these is yours to write

*in what order?
none*

*who locks it?
you do*

*did it land?
nothing says*

*when do I look?
you decide*

a file is a message: a batch of them, in a channel that happens to be a filesystem

Shared Database

Both applications read and write the same schema, typically through an ORM.

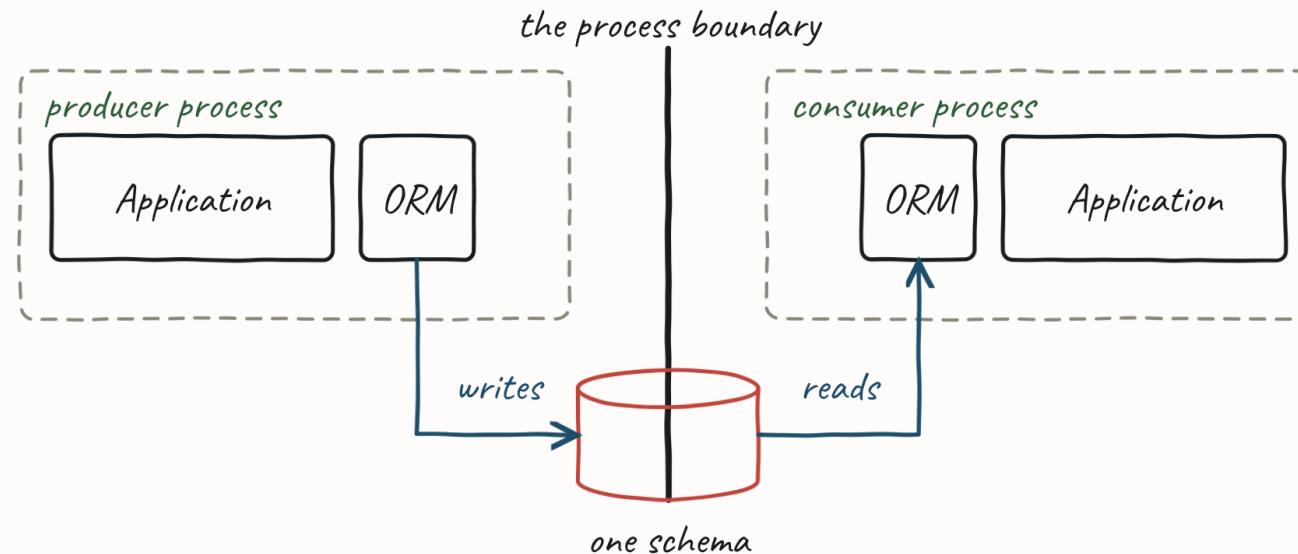
- **Asynchronous conversation** — the writer and the reader need never be present at the same time.
- **Common coupling** — and that is the problem.

Nobody owns the schema. A change propagates to every reader whether they wanted it or not, and you are back to agreeing release dates across teams — which is exactly what the boundary was for.

Shared Database

| This is the one style that hands back what the boundary bought you.

nobody owns the schema – a change reaches every reader whether they wanted it or not



not temporally coupled – neither has to be up when the other is. what couples them is the shared mutable schema and that is the boundary itself, handed back: you are agreeing release dates across teams again

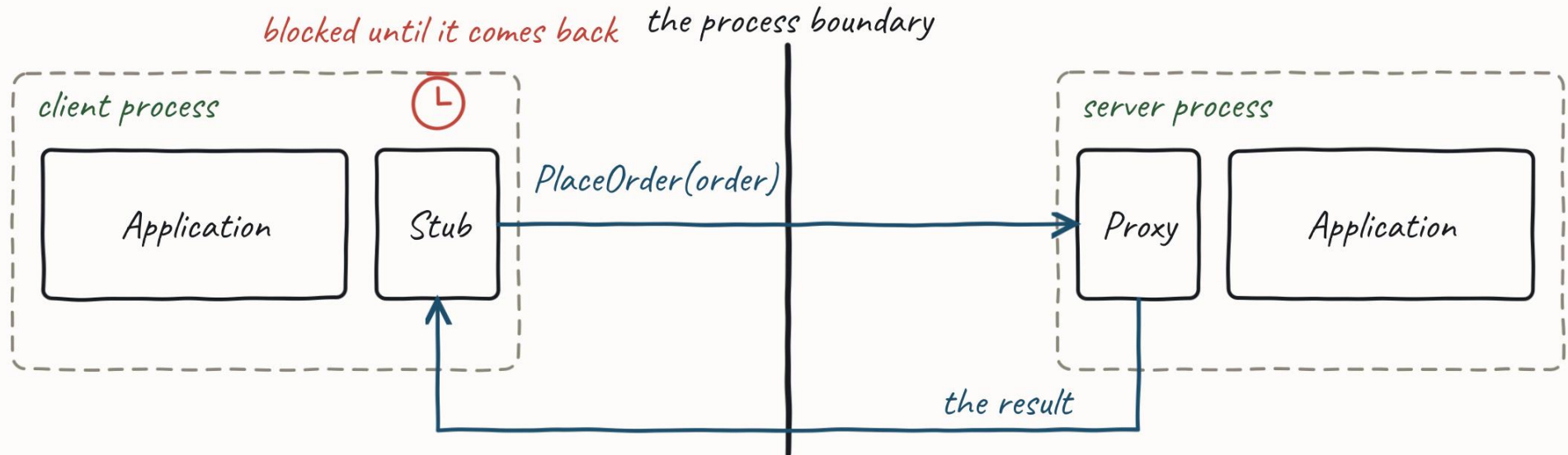
Remote Procedure Call

One application invokes an operation on another and waits for the result.

- **Synchronous conversation** — both parties must be up.
- **Control coupling** *and* **temporal coupling**.

Remote Procedure Call

the only style that loses on both axes at once – you say what to do, and you wait while it is done



*an operation name, not data – you are telling the other side what to do, which is control coupling
and both must be up at the same moment, so an outage here is not a delay, it is a failure*

Messaging

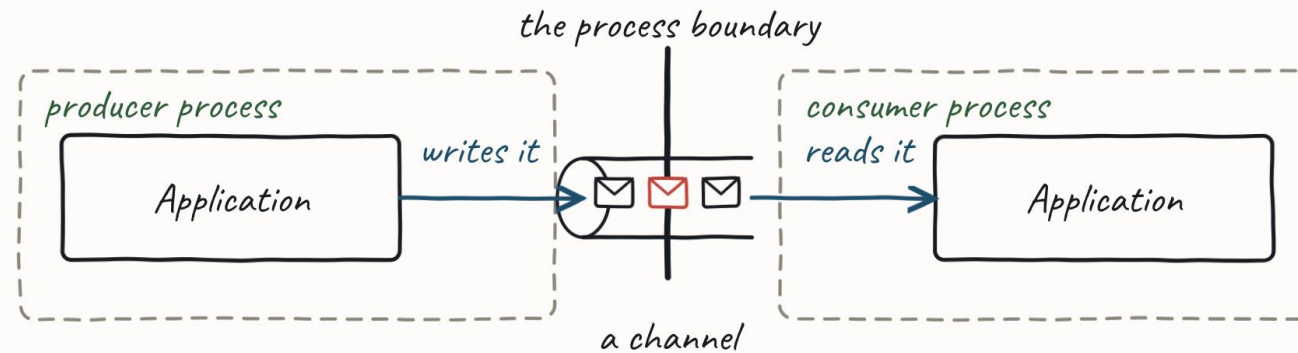
One application writes a message to a channel; another consumes it.

- **Asynchronous conversation** — store and forward; the outage becomes a delay.
- **Coupled about... whatever you put in the message.** A command is control coupled. A whole-entity event is stamp coupled. A message carrying only what the receiver needs is data coupled.

Messaging

| Messaging is the only style where the coupling is a decision, not a property of the style.

the coupling is a decision here, not a property of the style – it is whatever you put in the message



the same four questions – and the broker has answered them

*in what order?
the channel*

*who locks it?
the broker*

*did it land?
an ack*

*when do I look?
poll, or be pushed*

the other three styles fix where you sit on the coupled-about axis; this is the one that leaves it open

Why Messaging

The four styles on the grid from §Coupling:

Style	Coupled <i>about</i>	Must we both be <i>up</i> ?	What it hands back
File Transfer	Data	No	Nothing — but you get nothing either
Shared Database	Common	No	The boundary itself
Remote Procedure Call	Control	Yes	Independent availability
Messaging	Your choice	No	Nothing

File Transfer and Messaging land in the same cell. Coupling does not separate them — so what does?

- **Ordering** — a directory has none. You write it yourself, or you get whatever the filesystem lists.
- **Locking and competing consumers** — two readers on one directory will fight. A broker makes that a configuration option.
- **Delivery guarantees** — did the reader finish? Did it crash halfway? Nothing in the file says.
- **Granularity and timeliness** — a file is a batch, and a batch arrives when the batch is ready.

Why Messaging

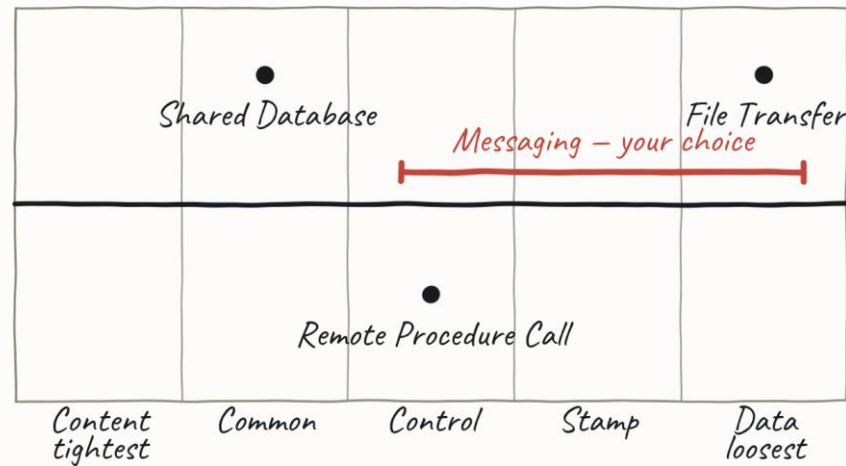
- | File transfer is messaging with everything useful left as an exercise.
- | Only one style keeps everything the process boundary bought you, and lets you choose the rest.

file transfer and messaging land in the same cell – only one of them can move

must we both be up?

*no –
both can be down*

*yes –
both must be up*



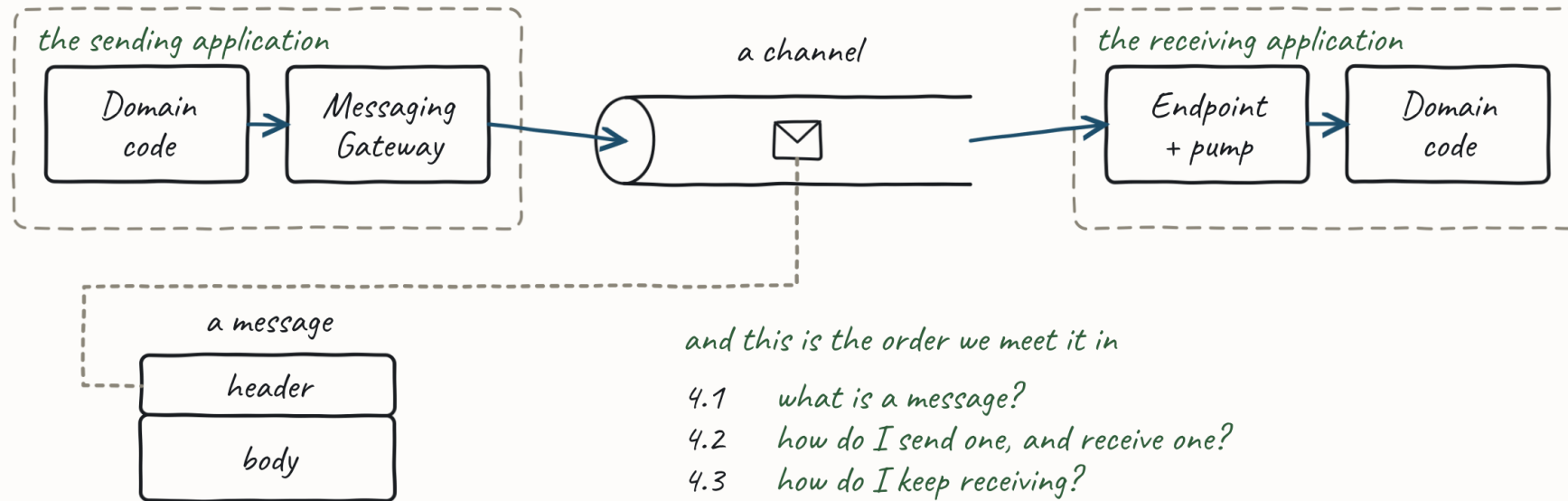
what are we coupled about?

Messaging Patterns

The Big Picture

A map of the messaging patterns we will cover across the day.

every pattern today hangs off this one line



and this is the order we meet it in

- 4.1 what is a message?
- 4.2 how do I send one, and receive one?
- 4.3 how do I keep receiving?
- 4.4 how do I stop losing them?
- 4.5 what kind of broker am I on?

4.1 What Is a Message?

Header and body, intent or fact — and whether the channel holds tasks or facts.

Message Construction

A message has a **header** and a **body**.

- The **body** contains data for the consumer.
- The **header** contains metadata for any filter in the pipeline, and should indicate the format of the body.
- Break a large message into pieces as a **Message Sequence**, or use a **Claim Check**.

Messaging vs. Eventing (Intent vs. Facts)

Two words the industry uses interchangeably, and they are not the same thing. After Clemens Vasters, two poles of message types.

	Messaging	Eventing
carries	intent	facts
which looks like	a Command (transfer of control), a Query (request an answer), a transfer of value	a notification — something you report on
expectations	part of a workflow, part of a conversation	none
concerned with	the future	the past — history, context

| A message asks for something. An event announces something.

Discrete vs. Series (Eventing Types)

After Clemens Vasters again, a second axis — and this one is about events only.

	Discrete	Series
handler	stateless	stateful partition processor
delivery	PUSH	PULL
position	independent	context, offset
shape	one thing, immediately actionable	continuous, sequential
what it says	something happened	a condition holds
part of	a conversation	a monolog

Command / Document / Event Messages

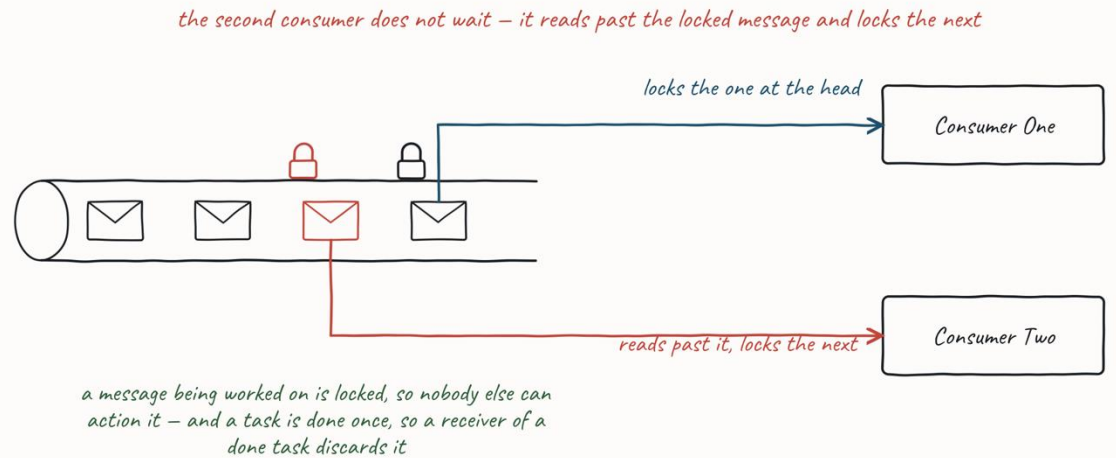
After Gregor Hohpe. Three core message types: **Command**, **Document**, **Event**.

- **Command Message** — reliably invoke a procedure in another application. Encapsulates a request as an object (GoF Command); usually sent Point-to-Point (one consumer).
- **Document Message** — reliably transfer a data structure between applications; the receiver decides what, if anything, to do with the data.
- **Event Message** — reliable, asynchronous event notification. The difference from a Document Message is timing and content — an event's contents are typically less important.

Queues Contain Tasks

A channel carries one of two things, and which one it is changes almost everything that follows. Messages on a **queue** are **tasks** — requests to carry out an action. Once the action is done, the task is gone.

- The first consumer **locks** the next message while it processes it.
- A second consumer **reads past** any locked message and locks the next available one.
- Nobody else should action a done task; a receiver of one discards it. If we cannot action it, someone else must.

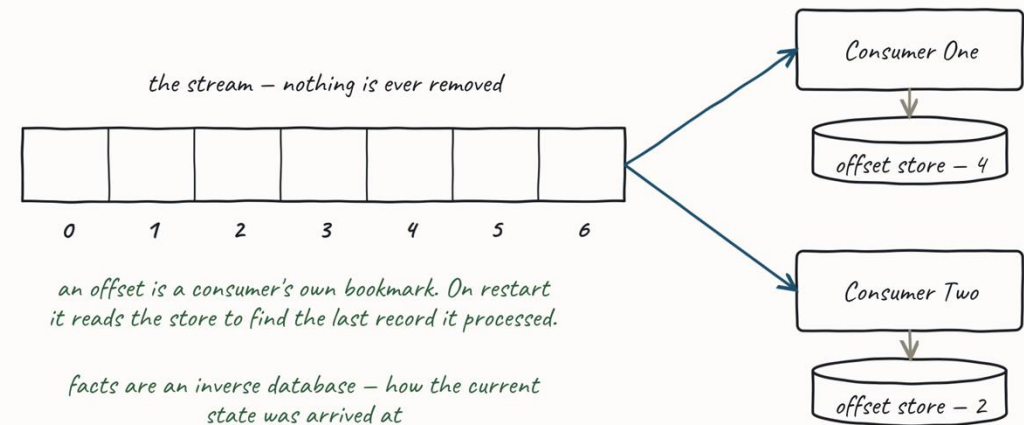


Streams Contain Facts

Records on a **stream** are **facts** — records that a state change occurred. Nothing is consumed by reading and nothing is deleted.

- The first consumer reads the next record and processes it; a second consumer reads the next and processes it — and **both can read the same records**.

nothing is consumed by reading — both consumers read every record, and each remembers how far it got



4.2 Sending and Receiving

How do I send a message, and how do I receive one?

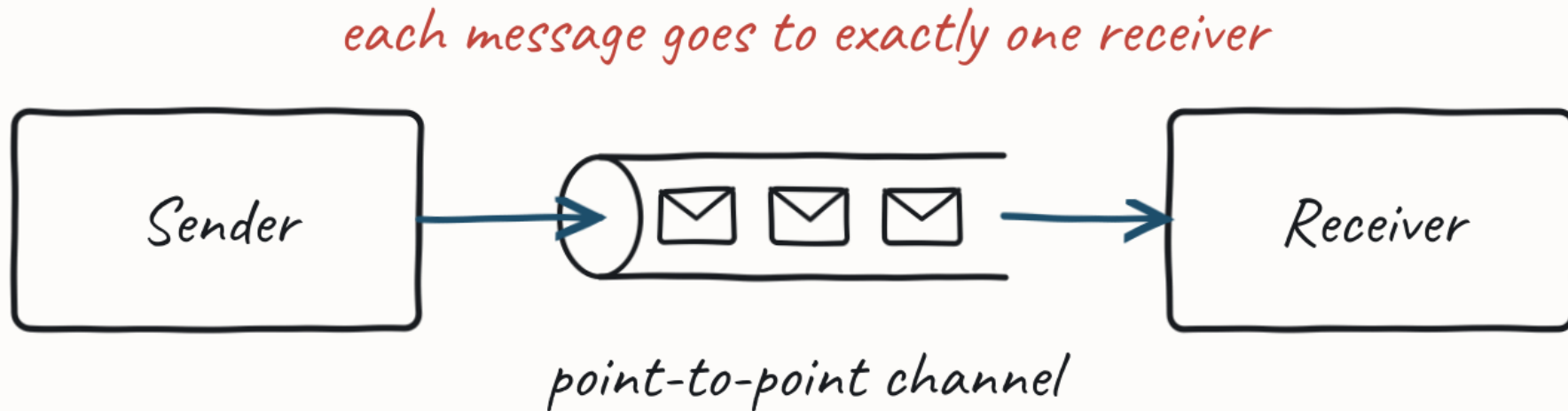
Channels

A **channel** is a virtual pipe that connects producer and consumer.

- A logical address (topic or routing key).
- Messaging is a "pipe", not a "bucket".
- **Unidirectional.** Two-way traffic is **two channels**, and the sender names the reply channel.
- One-to-One or One-to-Many.

Point-to-Point Channel

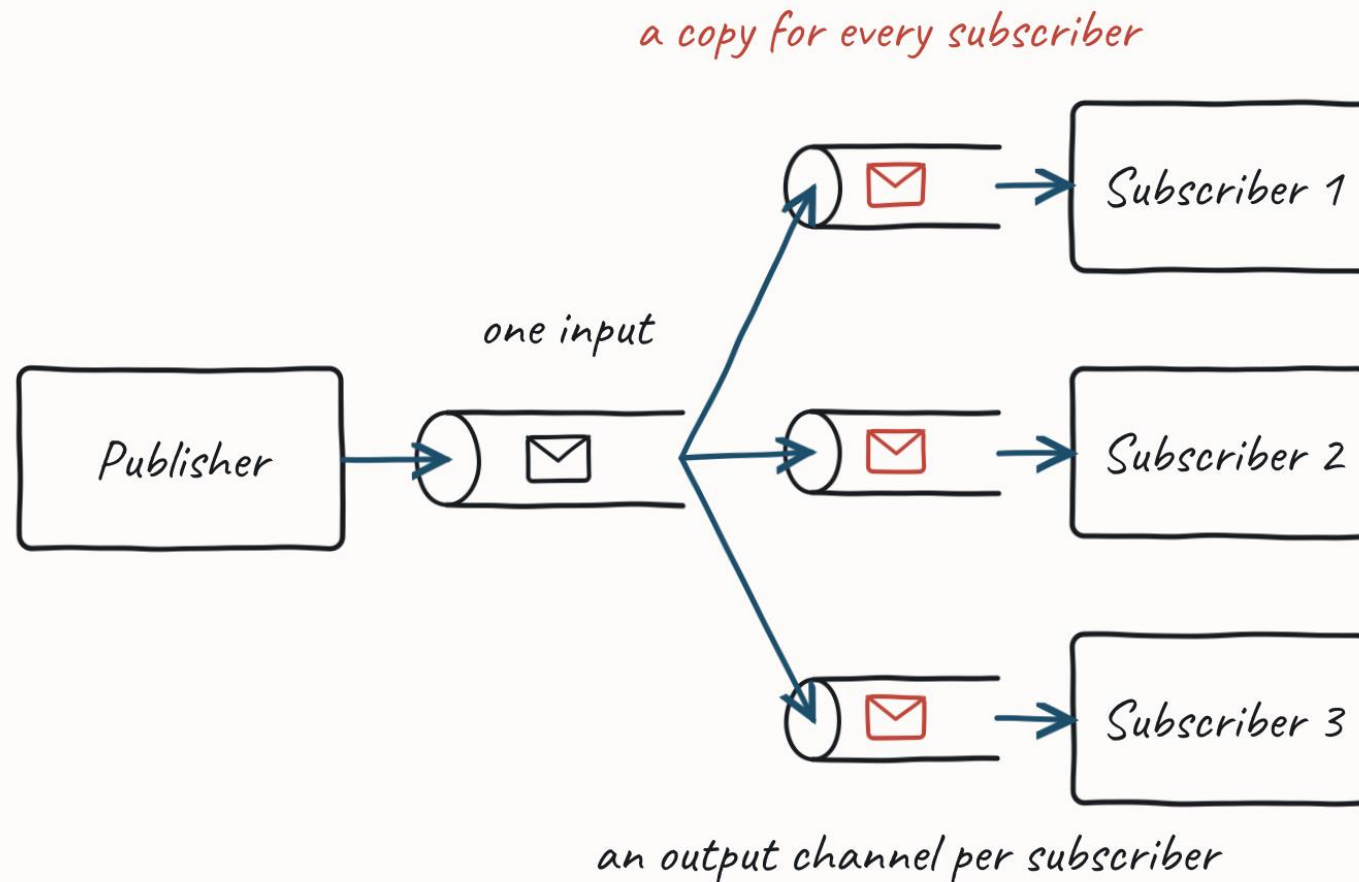
Only one consumer receives any message. With multiple consumers, the channel ensures only one succeeds, so receivers need not coordinate.



add receivers and they still do not need to coordinate

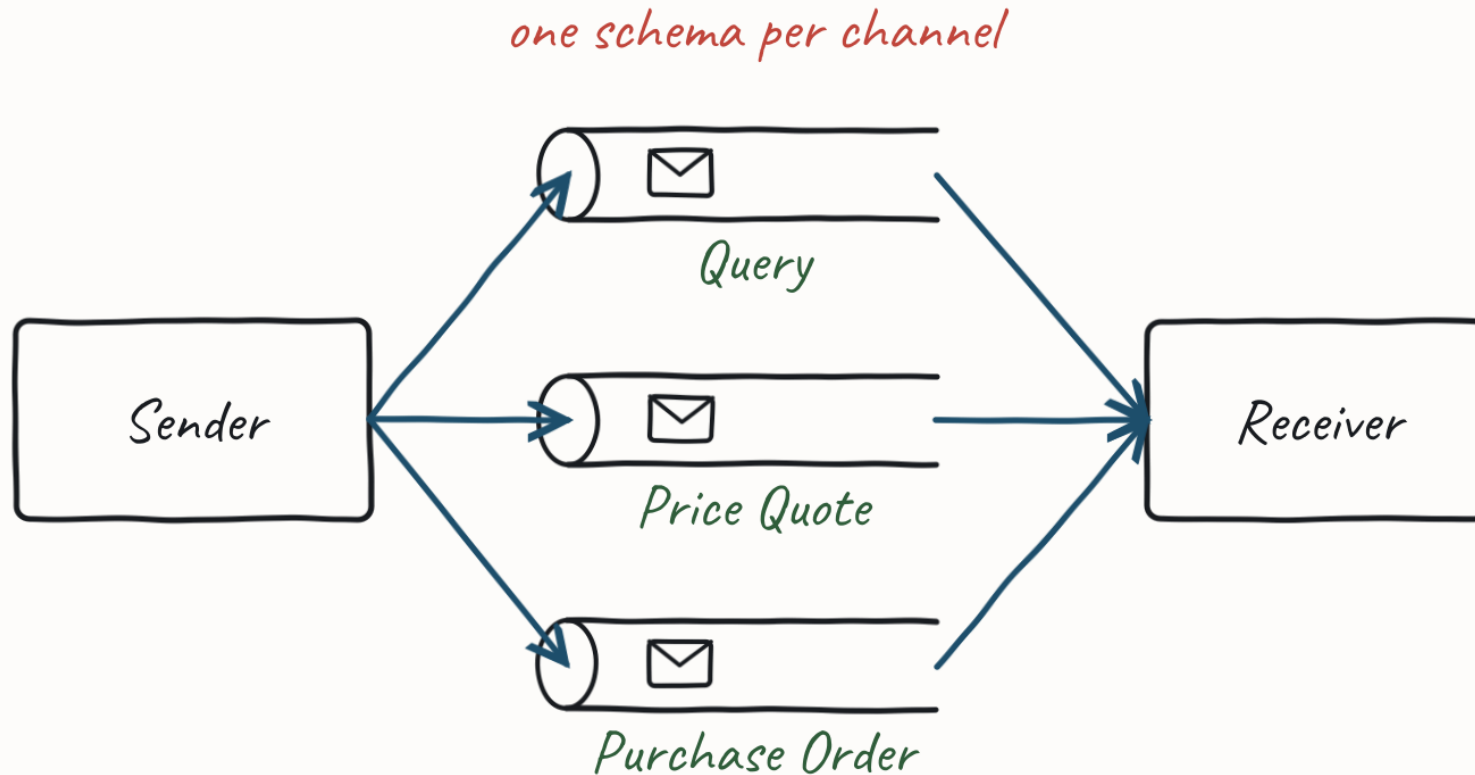
Publish-Subscribe Channel

Delivers a copy of each message to every subscriber. A publisher sends to one input channel, replicated onto many output channels, one per subscriber.



Datatype Channel

Use a separate channel per message schema so all messages on a channel share the same schema.

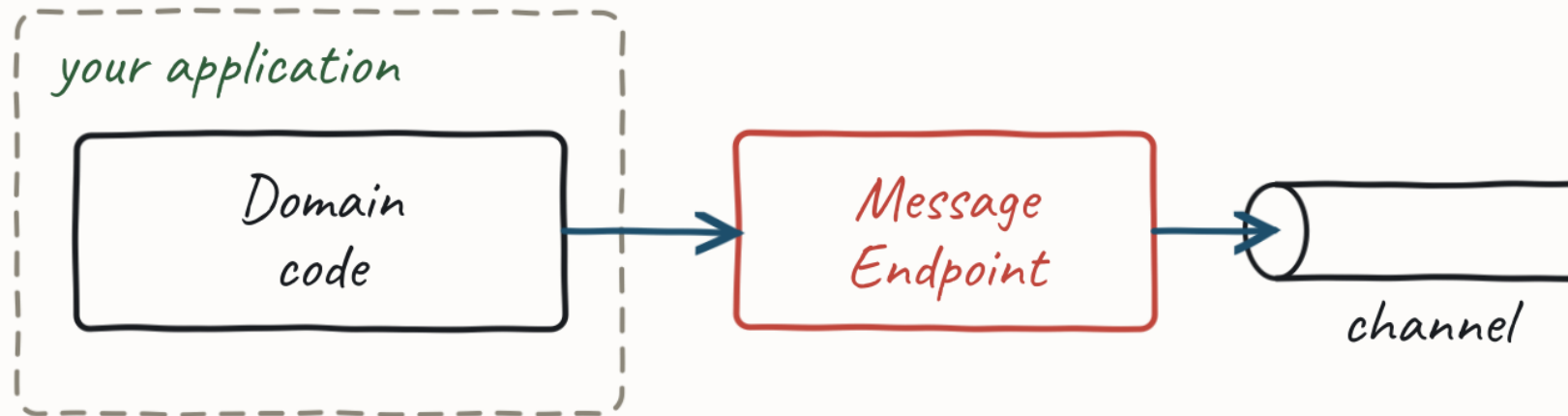


so the consumer never has to inspect a message to know how to read it

Message Endpoint

How does an application connect to a channel to send and receive?

*the endpoint is where the application
interoperates with others via messaging*

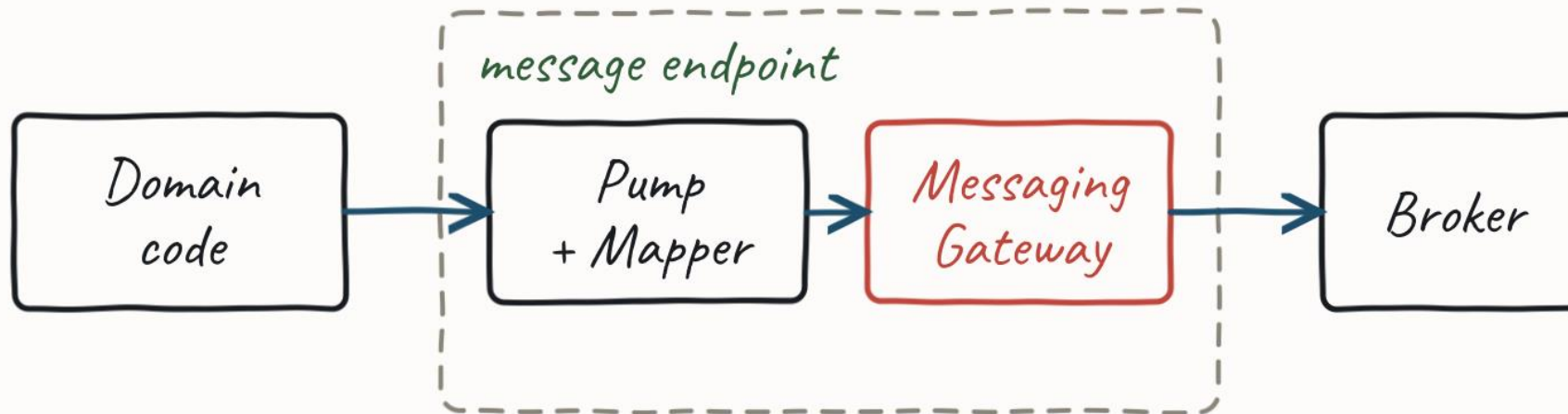


*it makes the message and sends it -- and on the way back,
takes the contents out and hands them over*

Messaging Gateway

Within the endpoint, encapsulate the middleware-access code in a **Messaging Gateway**, isolating all middleware dependencies into one component so the domain is isolated from messaging concerns.

the only component that knows which broker this is



swap the broker and nothing to the left of the gateway changes

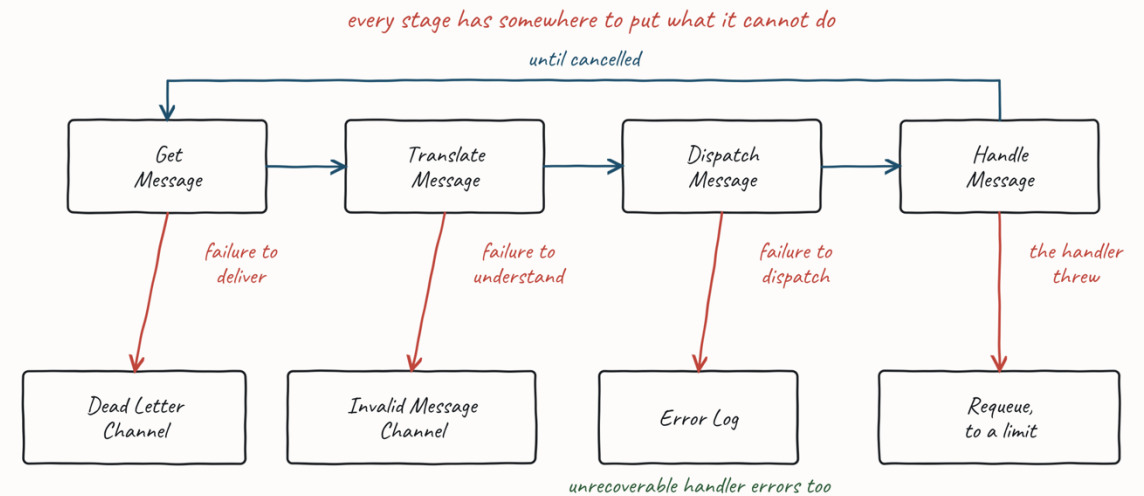
4.3 The Message Pump

How do I keep receiving — looping on the receiver.

The Message Pump

The code that takes a message from a channel and delivers it to application code, running in a loop until cancelled: **Get** → **Translate** → **Dispatch** → **Handle**.

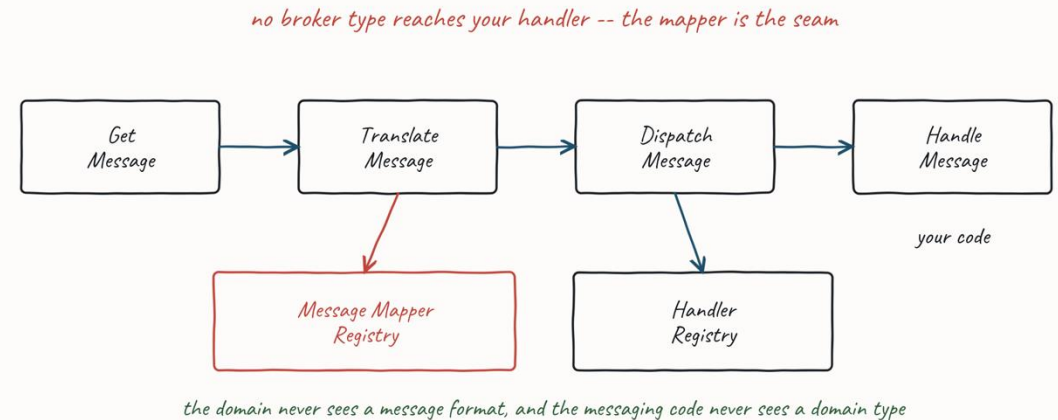
Each stage fails in its own way. That is why a message which is never going to be handled has four different places to end up.



Translate and Dispatch

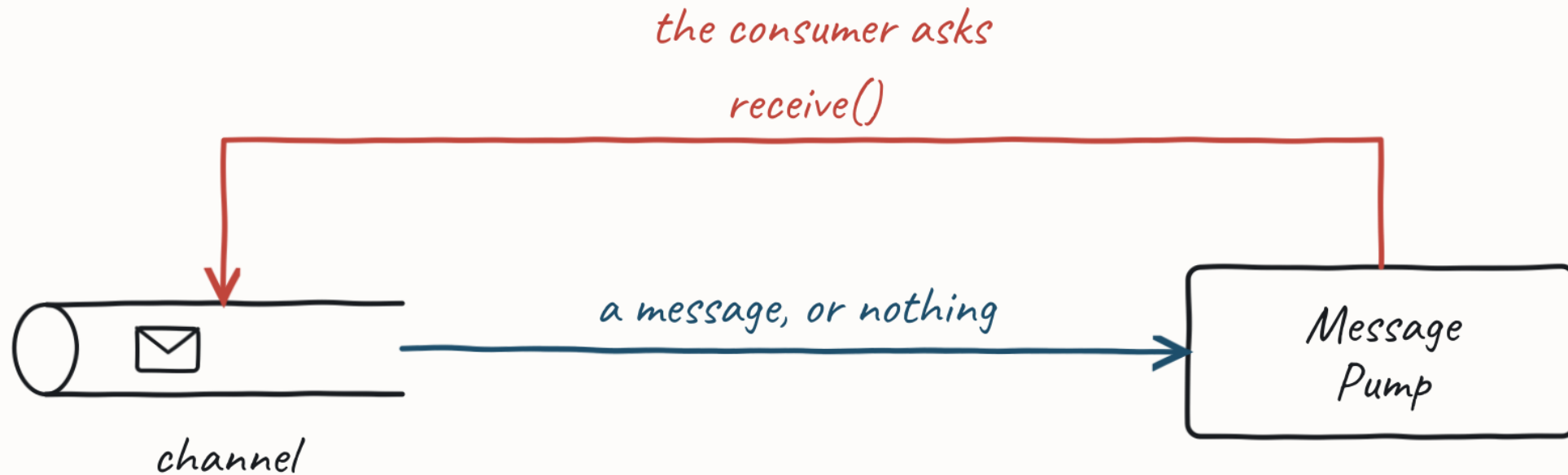
Two of those four stages are driven by a registry: **Translate** looks up a **Message Mapper**, **Dispatch** looks up a **handler**.

- A **Message Mapper** converts between a message and a domain object.
- A **handler** is your code, subscribed to the channels the endpoint listens on.



Polling Consumer

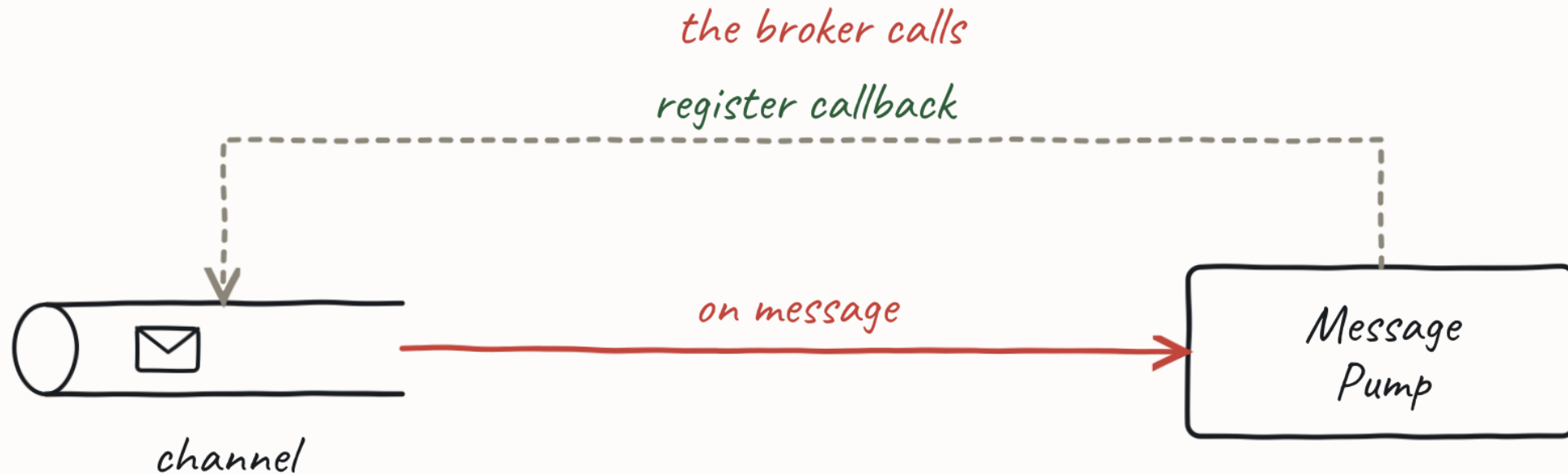
The message pump makes an explicit call to check for messages. Consumes a thread even when idle, but needs no held-open connection to the middleware. (EIP reference.)



*holds a thread even when the channel is empty --
but needs no connection held open*

Event Driven Consumer

The pump registers a callback that the middleware invokes when a message is available. Doesn't consume a thread, but requires a held-open connection (or the app to serve middleware requests). (EIP reference.)



*no thread while idle -- but the connection stays open,
and the broker sets the pace*

Service Activator

The line between the messaging code and your code.

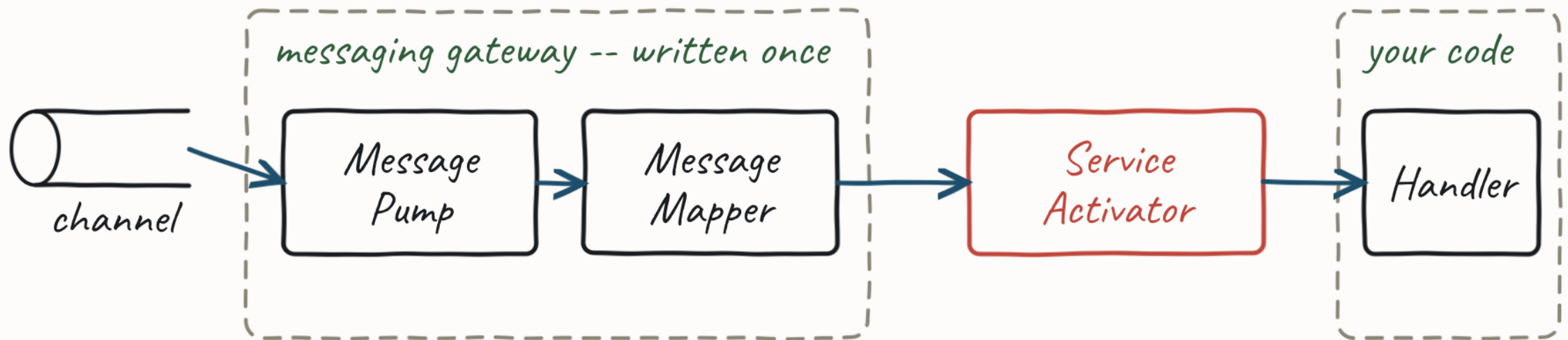
Everything in this sub-topic so far — endpoint, pump, mapper, registries — is the **messaging gateway**. The **handler** is your code. The Service Activator is the join, and its whole job is that the handler **does not know the pump exists**.

- The pump has a message; the **Message Mapper** has already turned it into a domain object.
- The activator makes an ordinary **synchronous, in-process method call** into your code, usually through a service layer.
- The handler takes a domain type and returns, or throws. No channel, no broker, no headers, no ack.
- **Dispatch** is hard-coded to one service, or reflective on the message type — which is what the *Handler Registry* on *Translate and Dispatch* is for.
- **One-way** (request only) or **two-way** (Request-Reply).
- **Because nothing in the handler is messaging-aware, anything else can call it too** — a developer test, an HTTP endpoint, a gRPC service. Useful, and worth calling out; but it is a *consequence* of the separation, not the reason for it.

Service Activator

| Your handler is not a message handler. It is a method that happens to be called by one.

the handler never learns it was a message



a domain type in, a return or a throw out -- no channel, no headers, no ack

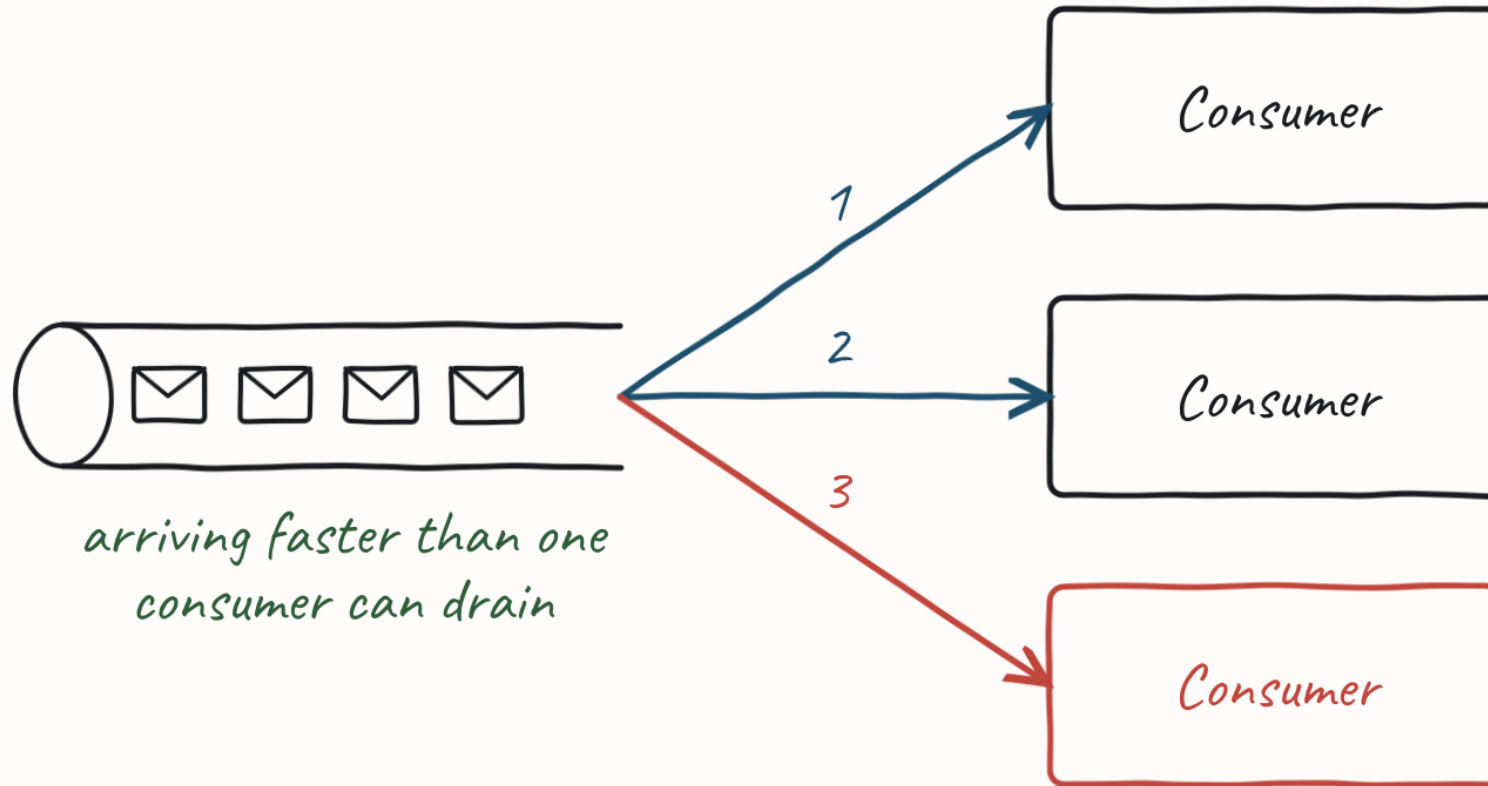
Competing Consumers

If the **Rate of Arrival** of messages (RoA) exceeds the **Rate of Consumption** (RoC), your channel backs up and the age of any message in it increases. At some point the time a message waits in the channel becomes unacceptable.

You solve this by **competing consumers** — but *the implementation differs between queues and streams*, and so does what it costs you.

Competing Consumers

add consumers until you consume faster than they arrive



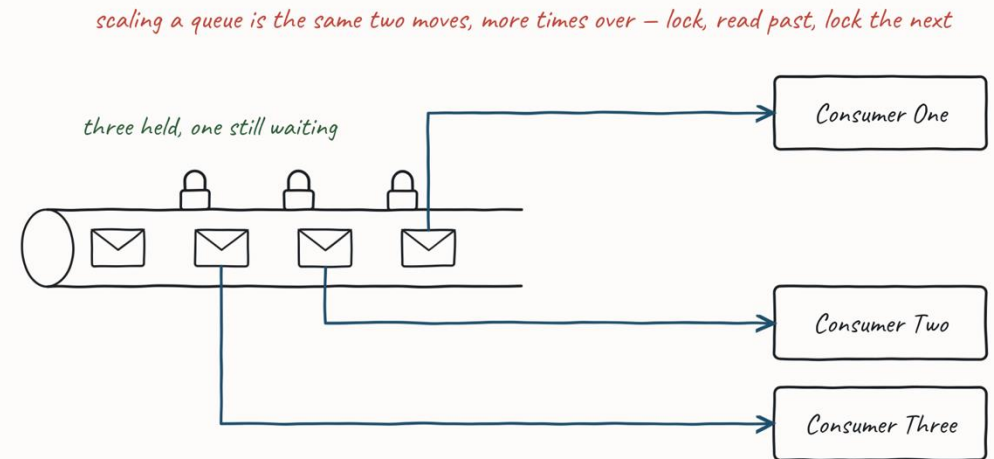
*arriving faster than one
consumer can drain*

the cost: lock-and-read-past means order is no longer preserved

Competing Consumers on a Queue

Add consumers to the same queue. The broker's **lock** does the rest.

- A consumer takes the next message and **locks** it — *this item is being processed*.
- Another consumer **reads past** the locked one and picks up the next item.
- Nothing says which of them finishes first, so **you have sacrificed ordering** to get the throughput.



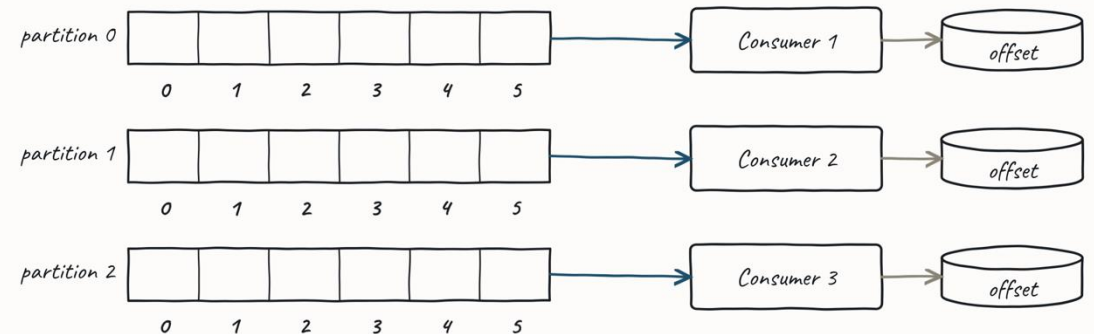
add a consumer and throughput goes up — nothing about the queue itself changes

Competing Consumers on a Stream

There is no lock, so there is no read-past. You scale by **partitioning**.

- **Partition by a key** — consistent hashing puts same-key records in the same partition.
- **Each partition has one consumer**, and each consumer keeps its own offset.
- Order holds inside a partition, so **you keep ordering** for everything that shares a key.
- The price: **no delay and no read-past**. A slow record holds up its partition, and there is nothing to skip it with.

splitting the stream costs you ordering — consistent hashing buys it back, one key at a time



each consumer manages the offsets for its own partition — there is no shared position

Worked Example — the Task Queue

The pump and competing consumers, as an application shape. **Offloading work from a request path.**

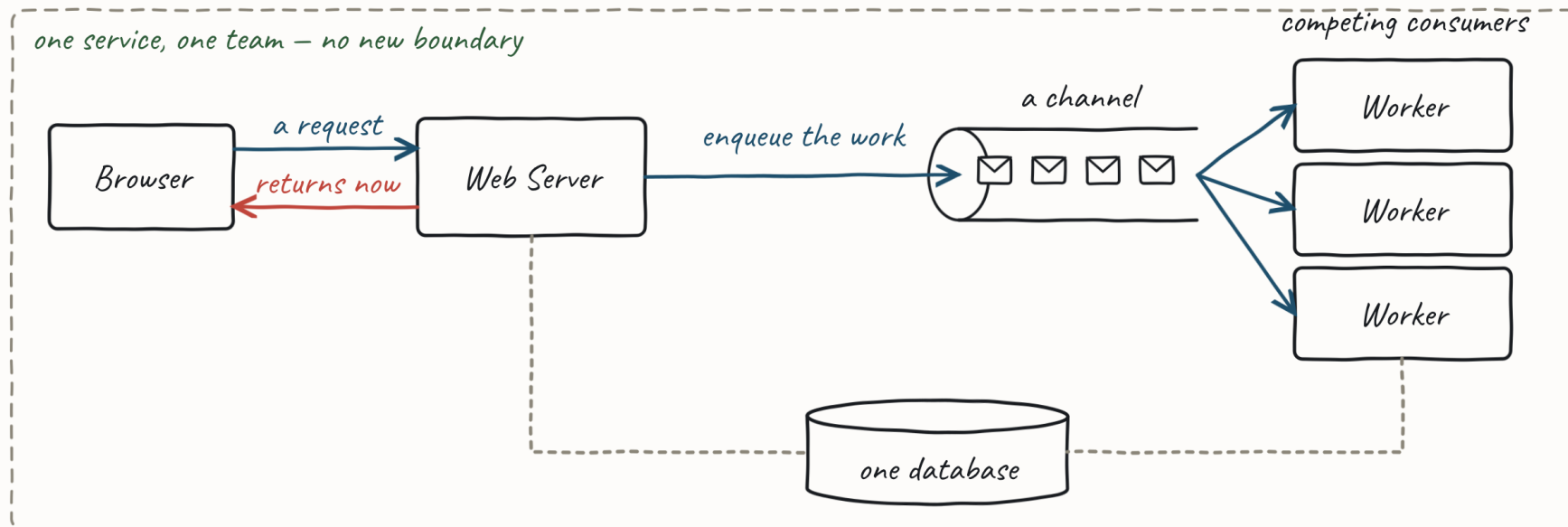
- The web server puts the work on a **queue** and returns immediately.
- The queue **stores** the work until we are ready to consume it — we can throttle to prevent surges.
- A backend application does the long-running or CPU-intensive work, freeing the web server to service new requests.
- We **scale out** the backend with competing consumers so the queue does not back up.

What it buys: the web server stays responsive when the backend is slow, overwhelmed, or down. The work is not lost — it waits.

Worked Example — the Task Queue

| This is the shape the opener promised. One team, one service, one queue — and here are its narts.

the web server returns before the work is done — and the work is not lost, it waits



*the web server stays responsive when the backend is slow, overwhelmed or down —
add consumers until you drain faster than the work arrives*

Task Queue — HTTP Flow

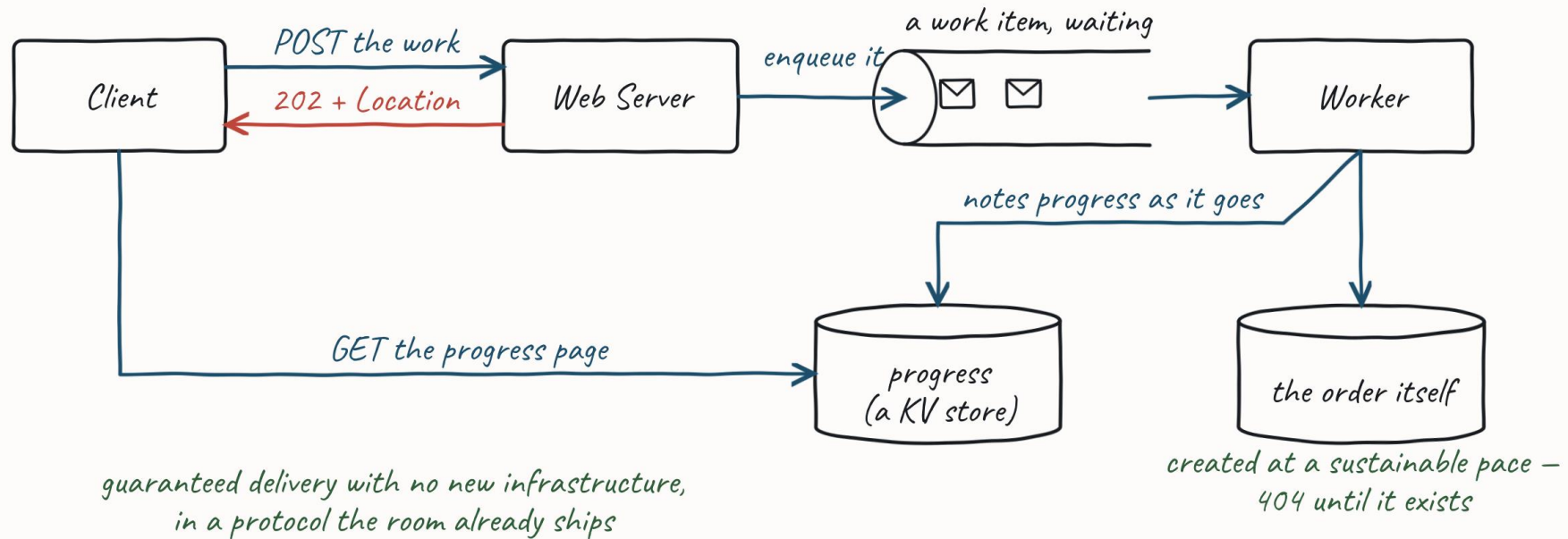
How this looks over HTTP — and most delegates have not used it.

- Return **202 Accepted** — we have your work request and we will not lose it. It is HTTP's own way of saying *store and forward*.
- Enqueue a work item for the request.
- Return a **Location** header so the caller can monitor progress: a link to the resource — 404 until it is created — and a link to a progress page backed by a KV store where we note progress.
- The backend does the work at a sustainable pace and updates the KV store as it goes.

Task Queue – HTTP Flow

| §4.4 is how you keep that promise.

202 says: we have your work, and we will not lose it



Exercise Material — Introduction & RMQ

Readme, videos, scripts & slides. Introduction to Exercises; Quick Start RabbitMQ — the AMQP 0-9-1 primitives (exchanges, bindings, queues) behind the channels of §4.2.

Then build the thing you have just been shown: a producer, and a consumer with a message pump — mapper, handler registry, and a handler that knows nothing about messaging.



DON'T PANIC

4.4 Guaranteed Delivery

How do I stop losing messages — on the way out, and on the way in?

Guaranteed Delivery

Coming from HTTP, a failed call is obvious: you get a status code, or a timeout, and you decide what to do. Messaging hides the failure. The send returns, and you find out later — or never.

Two halves, and you need both:

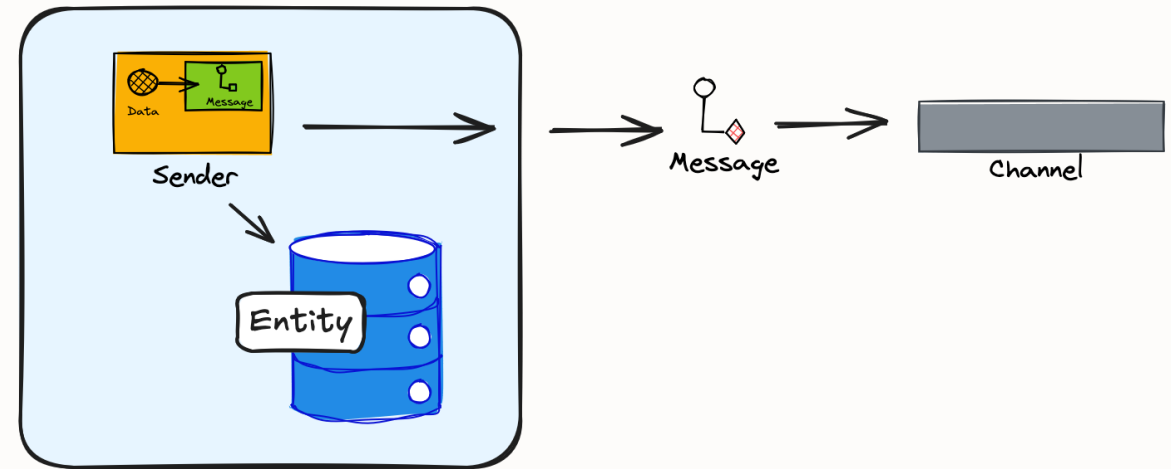
- **Producer side** — did the message actually get out, given the entity write and the send are not in the same transaction?
- **Consumer side** — what happens to a message my code cannot process, and how do I stop it taking the consumer down with it?

| "Sent" is not "delivered", and "delivered" is not "processed".

The Dual-Write Problem

No transaction spans both my write to the DB for an entity *and* my sending of a message.

- If the message send fails, downstream systems become inconsistent with upstream.
- If we reverse the operations and the message sends but the DB write fails, upstream is inconsistent.

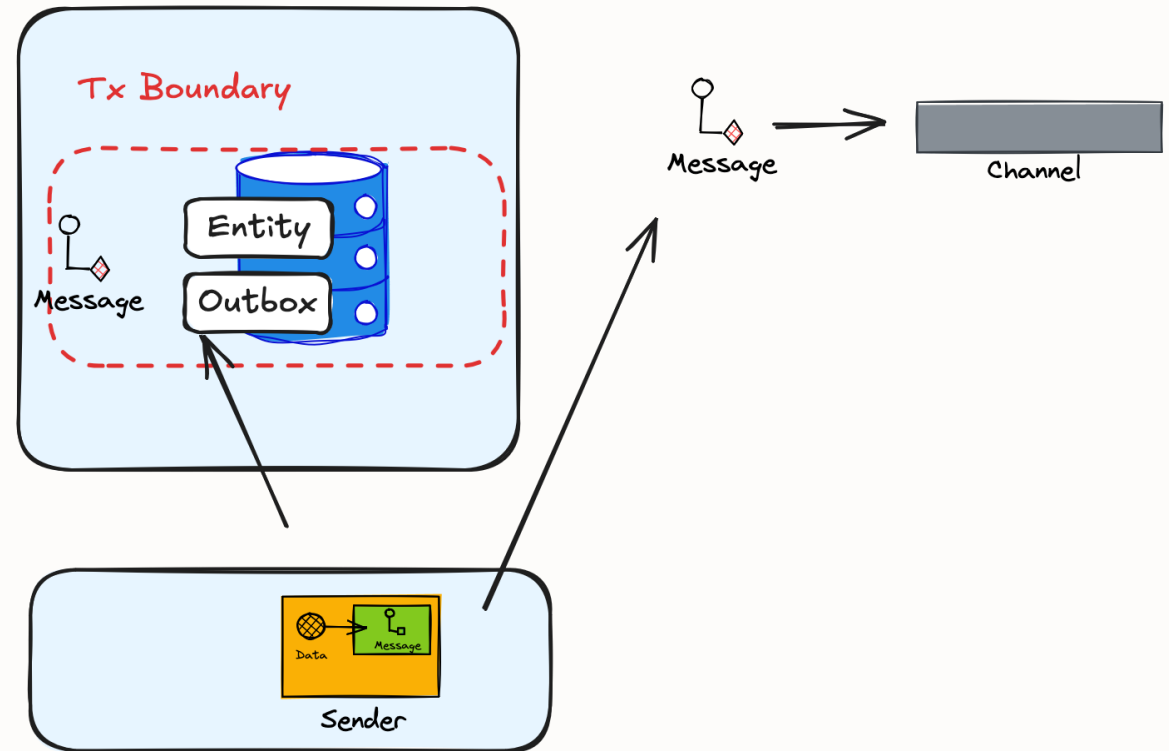


Outbox

Instead of writing directly to the channel:

- Open a transaction, update the entity, and write the message to send to an **Outbox** table as *Pending*.
- Once sent, mark the message *Sent* in the Outbox.
- A background "sweeper" runs at an interval to flush any Pending items (over an age) from the Outbox.
- To reduce latency, also send from the application right after writing, with the sweeper as backup.

The outbox guarantees at-least-once.
Which is a polite way of saying: you will send duplicates.



Log Tailing (Change Data Capture)

No ACID transaction across the Outbox and the entity tables? Tail the transaction log instead.

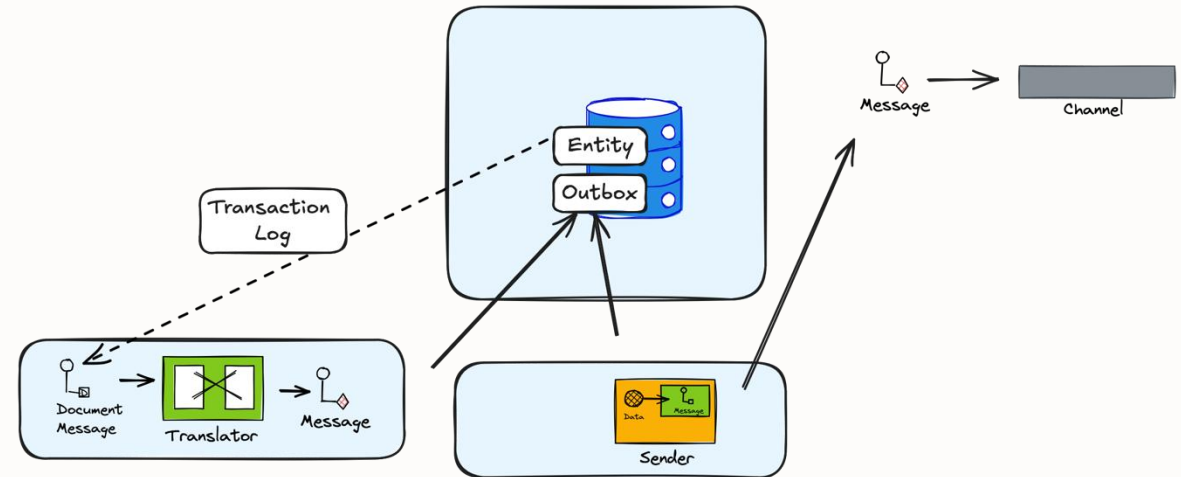
The naive version, and the one CDC tooling makes trivially easy:

transaction log → broker. Done.

The problem: **your table schema is now your published contract**. Every column rename breaks consumers you have never met.

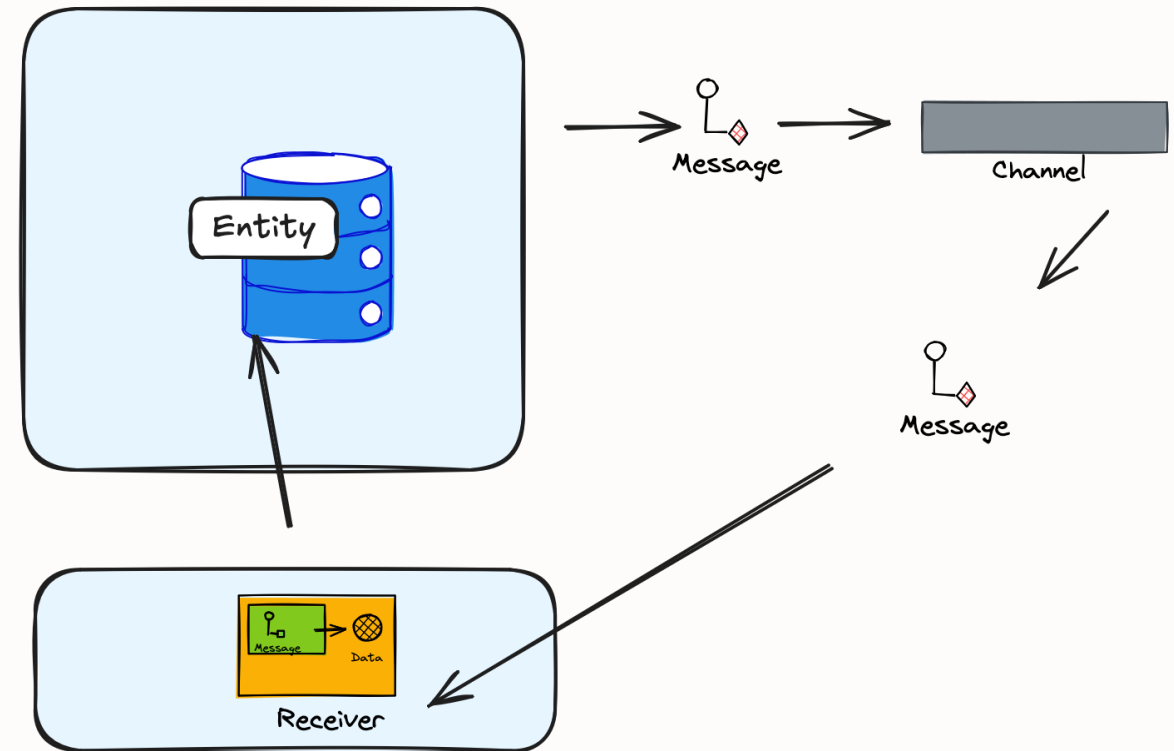
Instead, the log tail feeds an **anti-corruption layer**: read the log, run the **mapper**, write the *message* to the **Outbox**, then sweep as before.

Change Data Capture without an anti-corruption layer publishes your schema. The write to the Outbox *is* the layer.



State Change Capture

A low-cost alternative: send the message *before* writing the entity. If the message sends, delivery is guaranteed. Read the message back from the channel and then update the entity. The main issue: you must cope with **eventual consistency**, which isn't always simple.



When the Handler Fails — Ack and Nack

The pump reads a message and calls your handler. **The handler throws.** Now what?

The pump has exactly one lever: **acknowledge** the message, or don't.

- **Ack** — I am done with this. The broker may forget it.
- **Nack** — I am not done. The broker keeps it, and will hand it to someone again.

Ack too early and a crash loses the message. Ack too late and a crash reprocesses it. **There is no third option** — which is why at-least-once stops being a slogan on the consumer side.

Not Acking — Requeue or Reject

Not acking is two decisions, not one. **Which one you take is a policy** — you set it in advance, and most teams never do.

- **Requeue** — put the message back on the queue. Someone tries it again.
- **Reject** — do not process this message at all. **Delete** it on a queue; **skip** it on a stream.

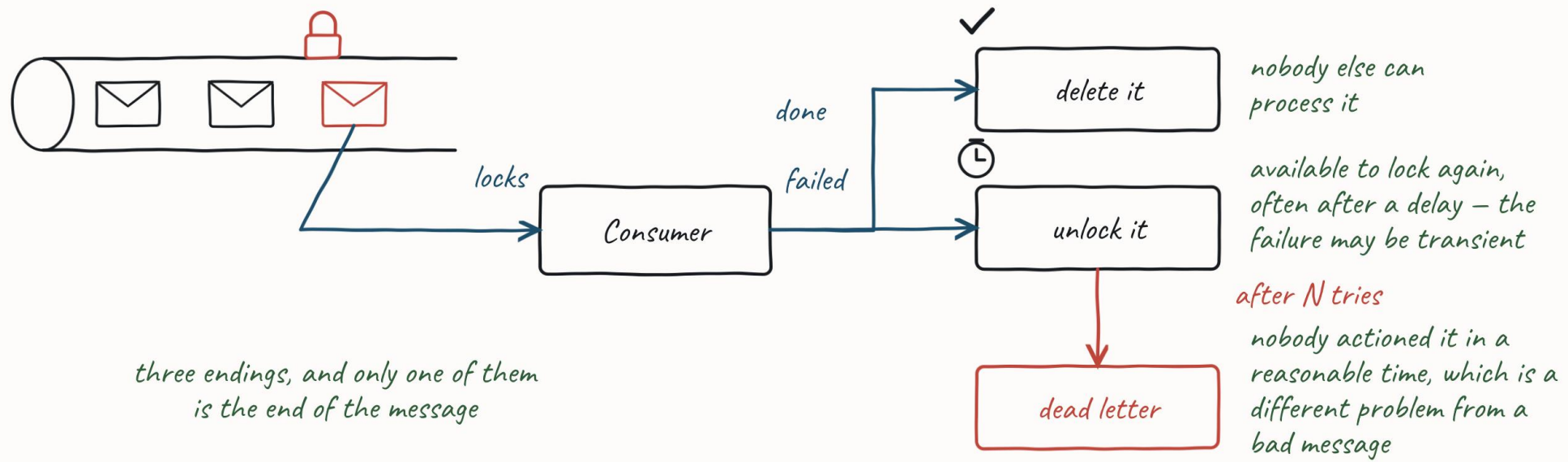
Each of the two has to end somewhere:

- On a **requeue**, set a policy for how many attempts. Exceed it and the message goes to the **Dead Letter Channel**.
- On a **reject**, a **badly formed** message goes to the **Invalid Message Channel**.
- A **well-formed** message you reject may go straight to the **Dead Letter Channel** — by policy, and it is your policy.

Not Acking – Requeue or Reject

A message that is only ever nacked blocks the queue forever. Requeue and reject both have to end somewhere.

unlocking is not the interesting part – what matters is whether anyone else may try again

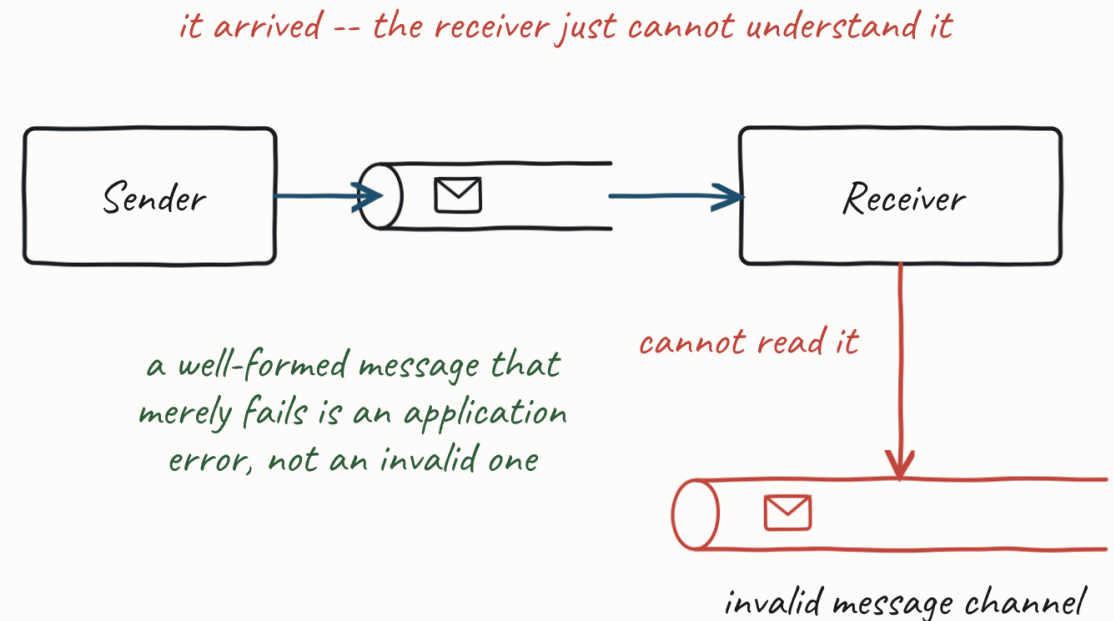


Invalid Message Channel

What happens when a *delivered* message cannot be understood (missing headers, wrong content type, schema mismatch)?

Needs from the broker: the ability to take the message off the channel and put it somewhere else *without* pretending it was processed.

Dead Letter: we could not deliver it.
Invalid Message: we delivered it and could not read it.



Requeue with Delay — a Queue Capability

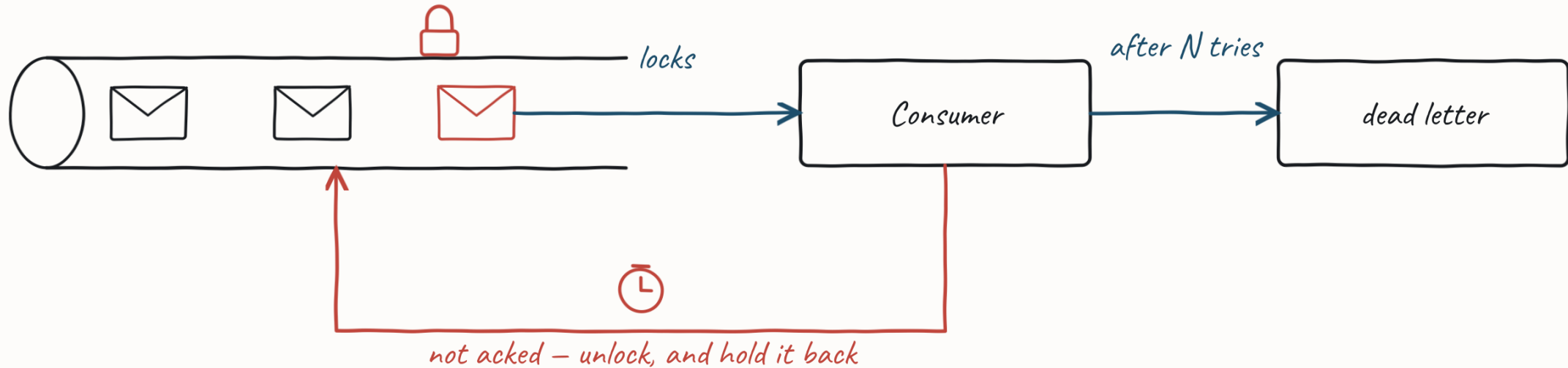
Transient failure is not the same as permanent failure. The downstream service is restarting; the database is failing over. The message is fine — you just tried at a bad moment.

- If work isn't done/acked, make it available again to the next consumer.
- If it failed for a transient reason, **delay** to let that pass.
- After a number of re-queues, move to a dead-letter channel.

Needs from the broker: per-message acknowledgement, a redelivery mechanism, and a way to hold a message back for a period. **The lock is what supplies all three, and only a queue has one** — which is what the slide after Dead Letter Channel is about.

Requeue with Delay – a Queue Capability

a transient failure is not a bad message – it is the same message at a bad moment

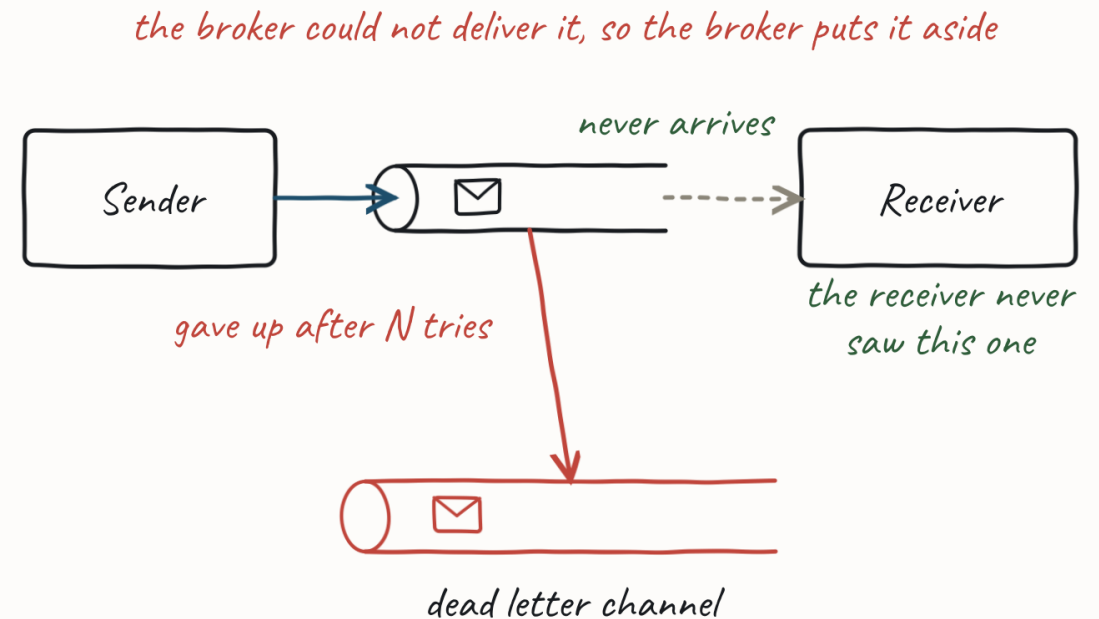


*per-message acknowledgement, a redelivery mechanism, and a way to hold a message back –
ask your broker for all three*

Dead Letter Channel

What does the middleware do with a message it cannot deliver to the intended channel? It may move it to a **Dead Letter Channel** for later operator review, often after retrying delivery a number of times.

Needs from the broker: somewhere to put the undeliverable message, and a rule for when to give up.



Streams — No Requeue or DLQ

Invalid Message Channel, *Requeue with Delay* and *Dead Letter Channel* all needed the broker to hold a message for you. A stream has no lock, so it has **no requeue**, no requeue-with-delay and no dead-letter channel. What you have instead:

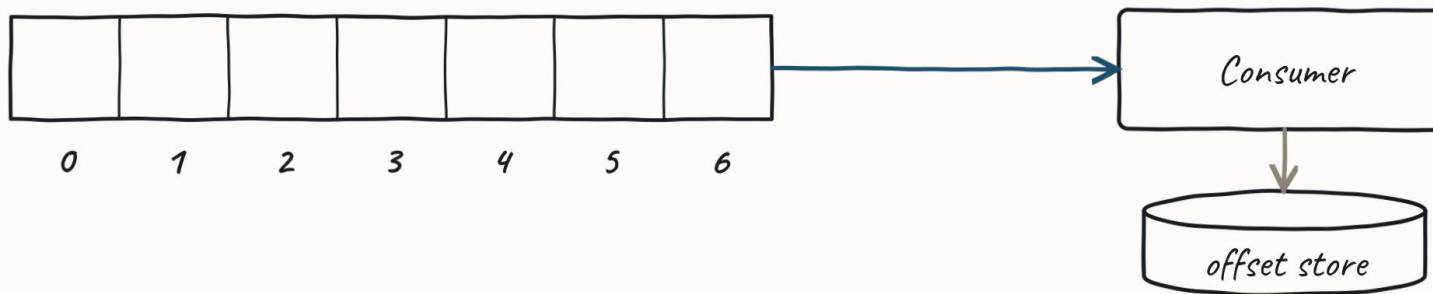
- **Ignore and continue** — load shedding. The record is gone and the partition moves on.
- **Retry in place** — backpressure. The partition stops until it succeeds.
- **Copy to another stream** — a delay stream or a DLQ stream, and a scheduler to feed it back.

The third one is the closest thing to requeue-with-delay, and it costs you the thing partitioning bought: the record comes back at the **end**. **You have de-ordered the stream to get a retry.**

Streams – No Requeue or DLQ

| On a queue the broker holds the message for you. On a stream, whatever holds it is code you wrote.

nothing is locked, so nothing can be put back – requeue and dead-letter are queue moves, and there are none here



✗ *requeue*

✗ *requeue with delay*

✗ *dead-letter channel*

there is no lock to release

and the offset has already moved on

nothing was taken out, to put anywhere

*instead: ignore and continue (shed load),
retry in place (backpressure), or copy
to a delay or dead-letter stream*

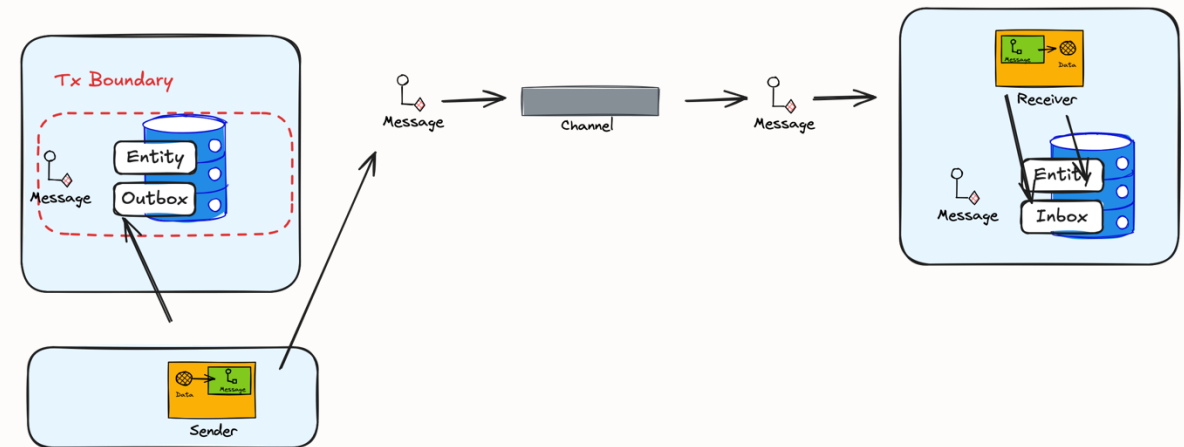
Inbox (Idempotency)

One more thing the pump has to decide, and this one comes from the producer side: have I seen this message before?

The Outbox bought at-least-once, so duplicates are not a risk — they are a certainty. This is where you pay for it.

If the message isn't idempotent (not side-effect free), use an **Inbox** to record messages *seen* (working on) and *processed* (may fail).

- Producer-side reliability creates a consumer-side obligation — the two halves are one design.
- If the handler *is* naturally idempotent, you do not need an Inbox. Most aren't.



What Your Broker Actually Gives You

Everything in this sub-section is a *broker* capability first. Frameworks backfill the gaps — which is fine, until you assume it is native and reason about failure as though it were.

Capability	RabbitMQ (queue)	AWS SQS (queue)	Kafka (stream)
Per-message ack / lock	Ack / nack	Visibility timeout	None — offsets only
Dead letter	Dead-letter exchange, native	Redrive policy, native	None — framework writes a separate topic
Delay / retry	TTL + DLX, or delayed-message plugin	Delay queues + per-message delay, native	None — framework uses retry topics
Redelivery count	Header, by convention	Receive count, native	None — framework tracks it
Ordering	Per queue	FIFO queues only	Per partition

| If your framework offers you a DLQ on Kafka, it built one. Know which of these you are relying on.

Exercise Material — Failing Well

Take the pump you built and make it survive the ways this sub-topic says it will fail: a message that **cannot be mapped**, and a message that **fails handling** n times. Use RMQ's own dead-letter exchange rather than building one.

Two different failures, two different answers
— and the pump has to tell them apart.



DON'T PANIC

4.5 Queues and Streams

The two things only one of them can do — and the summary you take away.

Scaling Streams — Consumer Groups

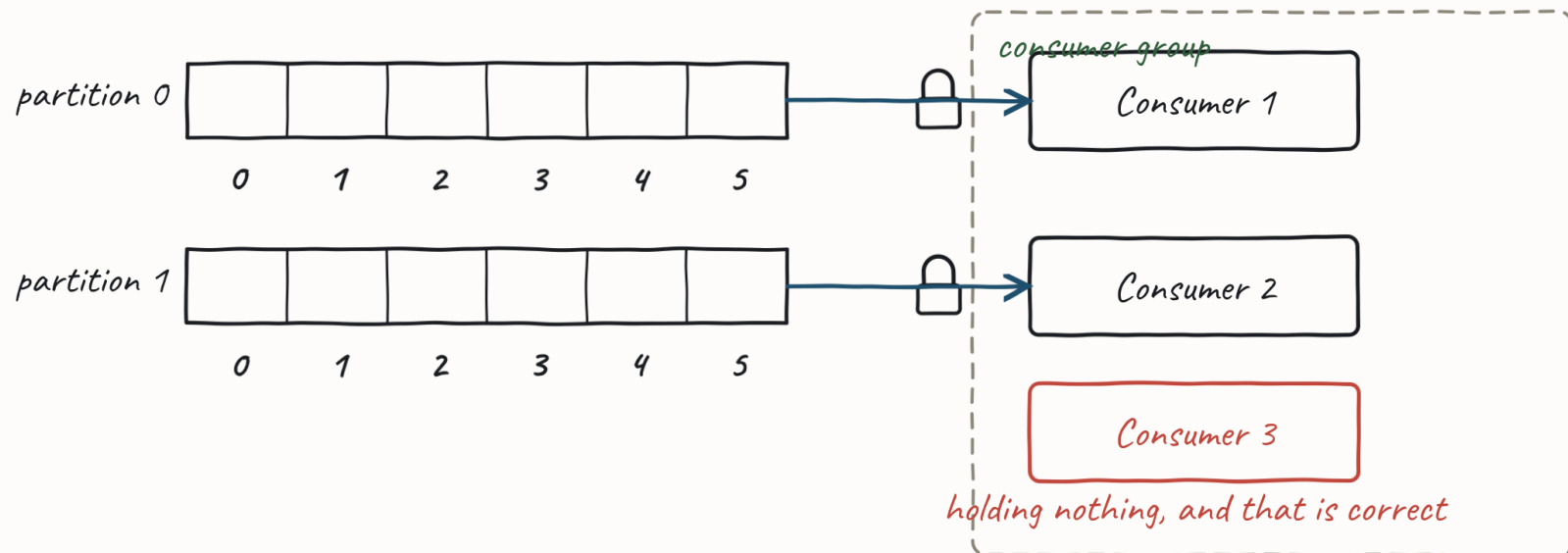
You partitioned the stream in §4.3 and gave each partition a consumer. A **consumer group** is how the broker keeps that assignment true while consumers come and go.

- Only **one consumer in a group** reads from a partition at a time — that is what protects the ordering partitioning bought you.
- A consumer may hold **more than one** of the group's partitions, so a group survives losing a member.

Scaling Streams — Consumer Groups

Partitions decide the ordering. Consumer groups decide who is holding them right now.

one consumer per partition at a time — the spare is there to take over, not to help



a consumer may hold more than one of the group's partitions — but a partition is never held by two at once

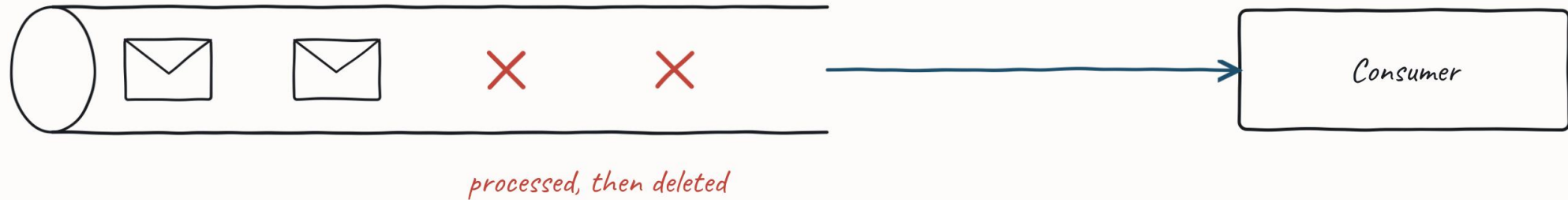
Archive and Replay

Can we re-read the past? The two models answer differently, and the answer follows from what §4.1 said each one **is**: a queue holds work to be done, and work that is done is gone.

Queues — No Archive and Replay

With queues we delete a message once the action completes, so there's no way to replay a work request — our only option is to ask the producer to resend.

the message is gone, so there is nothing to replay — your only option is to ask the producer again

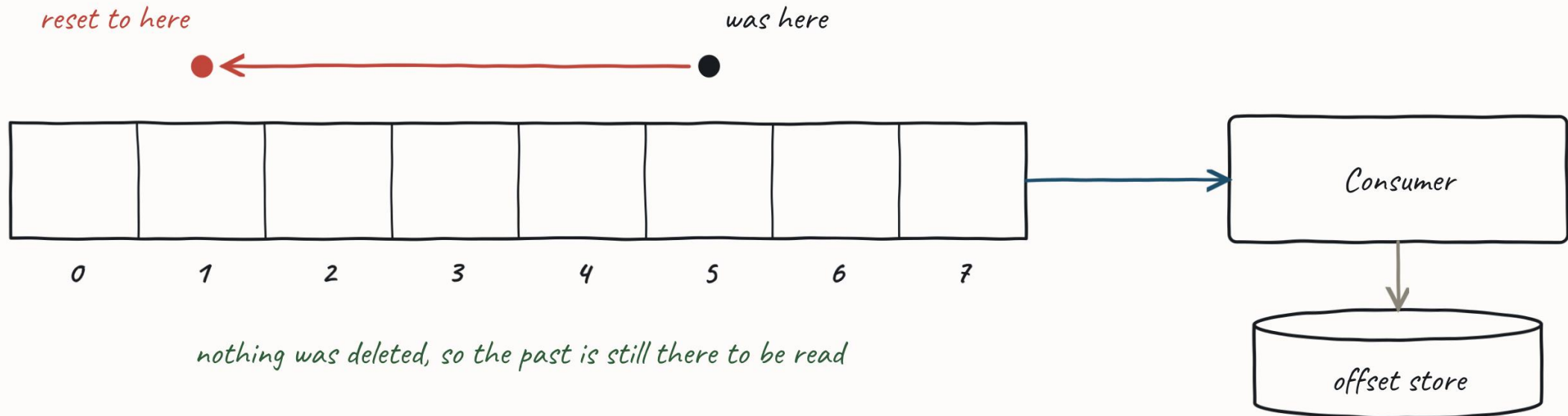


a queue holds work that has not been done yet — it was never a record of anything

Streams — Archive and Replay

Straightforward, because nothing is deleted: reset the consumer's offset to re-read the stream.

replay is not a feature of the stream — it is a consumer moving its own bookmark backwards



Queues vs. Streams — Capability Matrix

Summary comparison across: Messaging (Discrete Event vs. Series Event), Ordering, Archive and Replay, and Requeue with Delay — for Queue vs. Stream.

neither column is the good one — every row you gain on one, you give up on the other

	Queue	Stream
<i>what it carries</i>	<i>a task — do this</i>	<i>a fact — this happened</i>
<i>ordering</i>	✗	✓
<i>archive and replay</i>	✗	✓
<i>requeue, and delay</i>	✓	✗
<i>lock and read past</i>	✓	✗

a queue is work that has not been done; a stream is a record of what happened. Everything else follows.

Exercise Material — Introduction to Kafka

Readme, slides. Quick Start Kafka — brokers, topics, partitions, consumer groups, offsets.

Then the exercises: **take the reliability you built on a queue and get the same guarantees on a stream.**

Nothing you relied on in §4.4 is native here.



DON'T PANIC

Conversations

From single messages to conversations — choosing an exchange pattern.

Messaging Participants

You can now send a message, receive it, and not lose it. Real interactions are rarely one message — they are **conversations**, and the shape of the conversation is a design decision.

An application acts as either a **requestor** or a **provider**.

- Prefer *requestor/provider* over *producer/consumer* here, because they name the **role**: are you providing the API, or using it?
- A channel is **unidirectional**. **A two-way conversation needs two channels** — and that one fact forces every pattern in this section.

Example. *Provider (Store Information)* manages stores for our ecommerce site. *Requestor (Search)* registers a store and optimizes for fast lookup on key search terms. The message `changedstore()` indicates new store details.

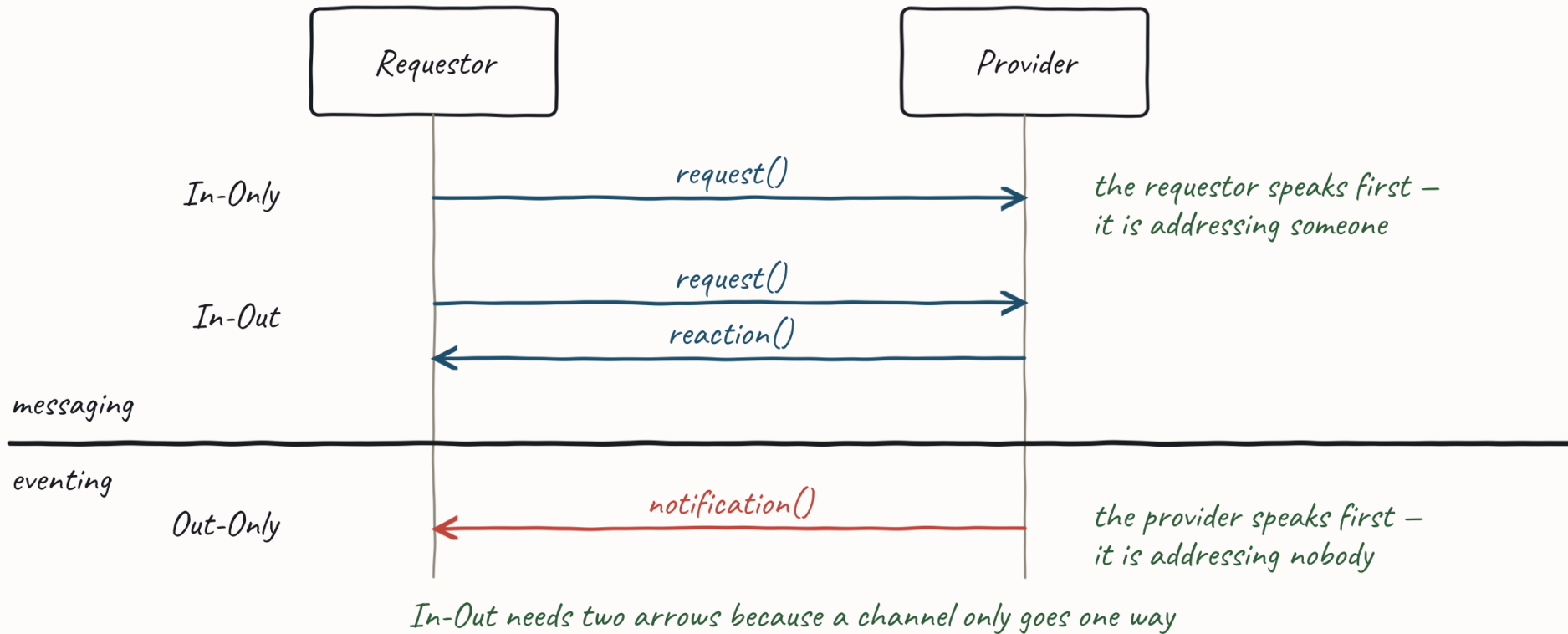
Messaging or Eventing?

The first question, before any pattern, and you met it in §4.1: **are you expressing intent, or reporting a fact?** Here is what it decides.

	intent — Messaging	a fact — Eventing
patterns available	In-Only, In-Out	Out-Only
who you are addressing	someone: you know who should act	nobody: the subscriber list is not yours
what you are coupled to	the request or command contract	the event schema

Messaging or Eventing?

the arrow turns round – and with it, whether you are addressing anyone



The Four Exchange Patterns

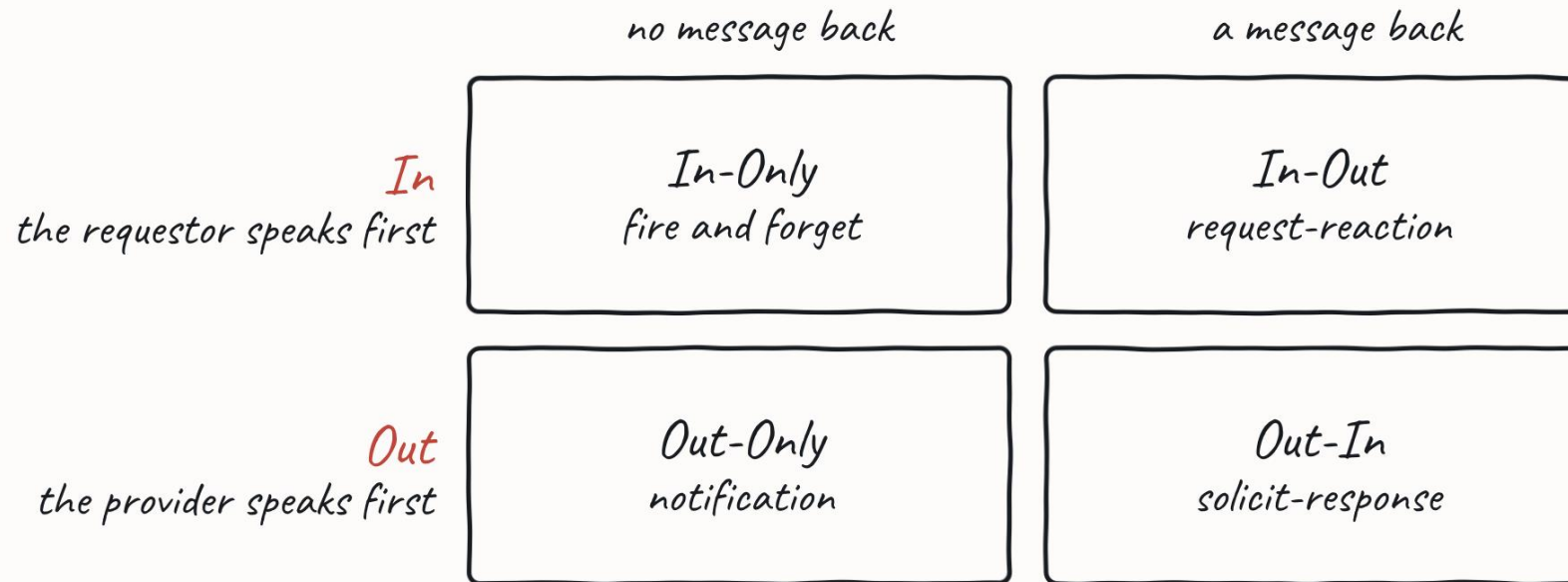
Two questions give you all four. **Who speaks first** — the requestor, or the provider? And **is there a message back?**

	no message back	a message back
In — requestor speaks first	In-Only (fire and forget)	In-Out (request-reaction)
Out — provider speaks first	Out-Only (notification)	Out-In (solicit-response)

- *In* and *Out* are named from the **provider's** point of view: In = a message arrives, Out = a message leaves.
- Everything else in this section is one of these four, or a composition of them.

The Four Exchange Patterns

In and Out are named from the provider's side – In = a message arrives, Out = a message leaves



everything else in this section is one of these four, or a composition of them

In-Only (Fire and Forget)

Under **In-Only**, the requestor sends a request to the provider but does not seek acknowledgment of completion. Typically called *fire-and-forget*.

- Used where we are finished with our part in a flow and are **transferring control** — we need no response because we are done.
- **Example.** *Requestor (cashier)* has a paid-for basket it wants to turn into an order; *provider (order placement)* raises an order — `place_order()`.

What it commits you to

- *Coupled about:* the command contract — you name an operation on someone else.
- *Must you both be up?* **No.** Store-and-forward; the provider can be down.
- Behavioural (control) coupling: you decided who should act.

Out-Only (Notification)

Under **Out-Only**, the requestor subscribes to the provider; an operation is triggered by receipt of a message, but it does not acknowledge the message back to the provider. Out-Only is the Publish-Subscribe pattern: a provider is not aware of its consumers.

- **Example.** *Provider (Store Information)* manages stores and raises `changedstore()`. Multiple requestors subscribe: **Search** (fast lookup), **Availability** (factors for whether a store is open), **Pick-Up Locations** (where couriers pick up, and restrictions).
- Adding a fourth subscriber requires no change to the provider — that is the whole point.

What it commits you to

- *Coupled about:* the event schema, and nothing else. No operation is named.
- *Must you both be up?* **No.**
- The loosest coupling available — and, as *Faults, by Pattern* shows shortly, that is exactly why it has no fault path at all.

Subscribe-Notify

In-Out **composed with** Out-Only: a requestor asks to subscribe to a provider, and then notifications flow.

- `subscribe()` → the provider manages a list of subscribers → optional `confirm()` / `confirmed()` (note the role switch — the provider is now the requestor).
- Then Out-Only `notification()` messages flow; a `stop()` ends them.

What it commits you to

- Behavioural (control) coupling on the subscribe exchange — then the loose coupling of Out-Only for everything after it.
- The provider now holds **subscriber state**, which it did not in plain Out-Only.

In-Out (Request-Reaction)

Under **In-Out**, the provider receives a request on one channel and returns a response — from the triggered operation — on a *separate* channel.

- The requestor sets a **correlation id** (conversation id) in the request header; the provider returns it in the response header, so replies arriving over a separate channel can be matched to the request that caused them.
- **Example.** *Requestor (Basket)* sends `checkout()` — a command asking the cashier to price a basket. *Provider (Cashier)* takes payment for the agreed amount and turns the basket into an order, replying `successfulpayment()`.

What it commits you to

- *Coupled about:* both the request contract and the response contract.
- *Must you both be up?* **Not necessarily** — as long as you do not *block*. See Blocking In-Out.
- Behavioural (control) coupling, and now you hold **state**: something must remember what the correlation id refers to.

Command or Query?

Command-Acknowledgement and Query-Result are the **same exchange pattern** — In-Out. What differs is the **intent** of the message, and intent is what decides what you must design for.

	Command-Acknowledgement	Query-Result
the requestor asks	<i>do this</i>	<i>tell me this</i>
example	Basket → Cashier: <code>checkout()</code> → <code>successfulpayment()</code>	Basket → Delivery: <code>getdeliveryfees()</code> → <code>deliverfee()</code>
provider state	changes	unchanged
retry is	unsafe without idempotency	naturally safe

Both carry behavioural (control) coupling — you named the provider and the operation.

In-Out — When Nothing Comes Back at All

The requestor may not receive the expected response at all. What can it do?

- Set a **timeout** within which to receive a response.
- **Retry** if no response arrives within that window (`greet()` → `greet()` → `acknowledge()`).
- Because we might send twice, the provider operation must be **idempotent**, or the consumer must **deduplicate** already-seen messages.

Blocking In-Out (Request-Reply)

The requestor provides a unique **Reply-To** channel in the request header; the provider uses it for the response, letting the requestor look up the suspended workflow — so a correlation id is not required. **The requestor may block** on the Reply-To channel while awaiting the reply.

| Block, and you have rebuilt RPC on top of a message broker.

What it commits you to

- *Must you both be up?* **Yes.** This is the one pattern in the set that puts **temporal coupling** back.
- Availabilities multiply again — the thing we distributed to avoid, and spent Guaranteed Delivery breaking.
- You have paid the cost of a broker and kept the failure mode of a synchronous call.

When it is still the right answer: an interactive request where a human is waiting and there is no useful "later" — the caller cannot do anything with a response that arrives in ten minutes.

Out-In (Solicit-Response)

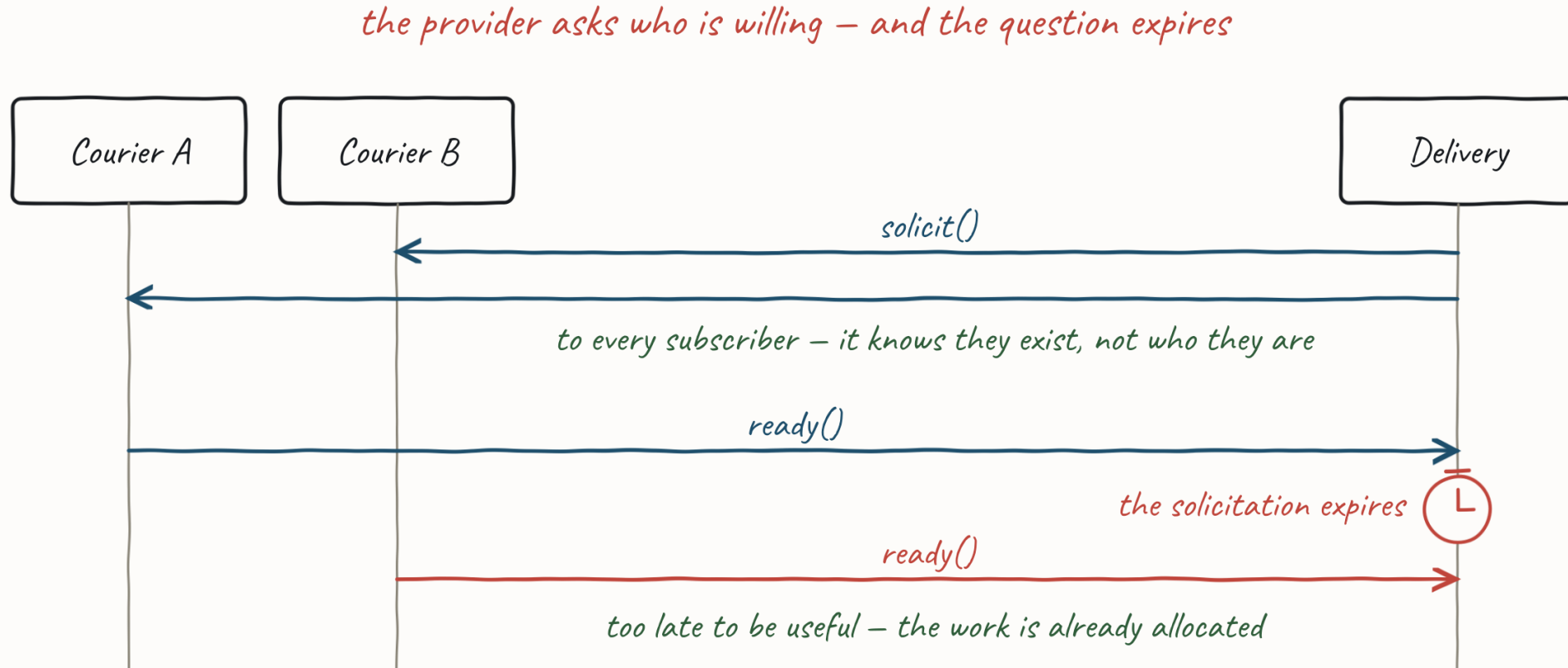
Under **Out-In**, a provider solicits a response from a subscriber and awaits confirmation; the subscriber confirms receipt of the provider's solicitation.

- **Example.** *Provider (Delivery)* assigns delivery requests to drivers and queries whether a courier is available — `solicit()` — usually followed by notifications of available work. *Requestor (Courier)* offers to take jobs depending on location and busyness — `ready()`.
- Use it when the provider needs to know **who is willing** before it allocates work.

What it commits you to

- *Coupled about:* the solicitation contract. The provider must know subscribers exist, though not who they are.
- *Must you both be up?* **No** — but the solicitation has a useful lifetime. An answer that arrives too late is worthless, which is a **timeout** problem, not an availability one.

Out-In (Solicit-Response)



a late answer is a timeout problem, not an availability one – both of you were up

Faults, by Pattern

You have met all four. **You do not get to choose their fault stories — the coupling already chose.**

pattern	what a fault can be	why
Out-Only	Nothing. No Fault is forced.	The provider does not know its subscribers. There is nobody to tell — and if it <i>did</i> know it would be coupled to them, and you would have traded away the property you chose pub-sub for.
In-Only	No Fault , or Message Triggers Fault (Robust In-Only) — a fault on a <i>reverse channel</i> , because there is no later message to replace	The requestor took no response, so a fault channel is an addition, not a substitution. Add one only if there is an action to take .
In-Out	Fault Replaces Message (Robust In-Out) — any message <i>after the first</i> becomes a fault instead of the normal outcome	There is already a response channel and a correlation id. The fault is a response , not an exception: same channel, same id, same code path.
Out-In	Fault Replaces Message , as for In-Out	Roles reversed, and nothing else — the provider waits.

| You cannot have loose coupling and a fault path back. Pick one.

| The question is never "should we handle faults?" It is **"is there an action the requestor would take?"**

Choosing an Exchange Pattern

The decision, in order:

- **Intent or fact?** Fact → Out-Only, and accept No Fault. Intent → keep going.
- **Do you need an answer?** No → In-Only. Yes → In-Out.
- **Who starts?** If the provider needs to canvass its subscribers → Out-In.
- **Do you need the answer *now*?** Only then Blocking In-Out — and know what you just paid.

pattern	coupled about	both up?	fault path
Out-Only	the event schema	no	none available
In-Only	the command contract	no	reverse channel, if there is an action to take
In-Out	request + response contracts	no, unless you block	fault replaces the response
Out-In	the solicitation contract	no	fault replaces the response
Blocking In-Out	request + response contracts	yes	fault replaces the response

Choosing an Exchange Pattern

only Blocking In-Out sits below the line – every other pattern lets both sides be down

must we both be up?

*no –
both can be down*

*yes –
both must be up*

		● In-Only ● In-Out ● Out-In		● Out-Only
		● Blocking In-Out		
Content tightest	Common	Control	Stamp	Data loosest

what are we coupled about?

6.1 Fat and Skinny Messages

Normalising the message: inline it, reference it, or replicate it.

What Goes in a Message?

The decision is **per field, not per message**. For every piece of data the provider needs in order to act, there are three choices:

- **Inline it** — put the value in the message.
- **Reference it** — put an id in the message; the provider looks the data up.
- **Replicate it** — the provider already holds a copy, kept fresh out-of-band.

| Designing a message is a normalisation problem.

- A **fat** message is fully denormalised. A **skinny** message is fully normalised. Neither extreme is usually right.
- The rest of this section is the rule that decides per field, and the price of each answer.

Fat Message

With a **Fat Message**, the requestor sends across all the *external* information a provider may need to perform the operation.

- The requestor provides all external information the provider needs to act.
- The resulting document message may be large (a **Claim Check** can help).
- **Example.** *Order Fulfilment* → *Courier Assignment*: the request provides delivery and pickup address, size, weights, etc. of the order.

What it buys you: the provider needs nobody else in order to act. No lookup, no cache, no second system that has to be up.

Fat Message — Transitive Dependencies

A purchase-order message that inlines customer and restaurant data shows the cost.

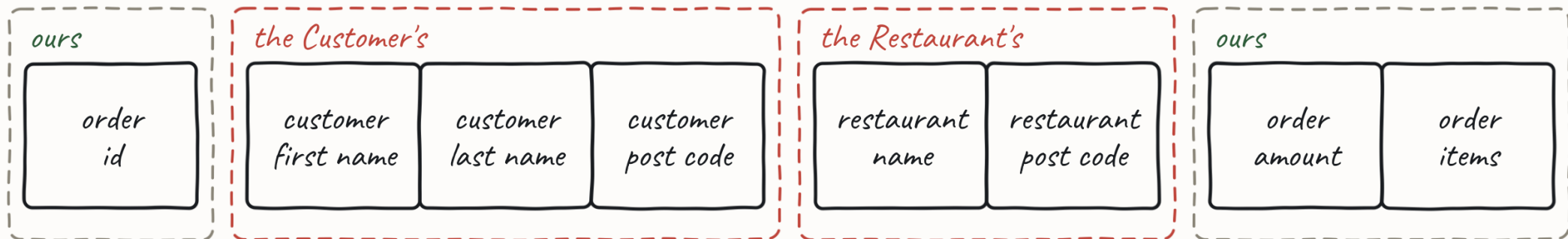
- The order data has the **lifetime of the message** — its schema changes if the purchase order changes. That is fine; it is *our* data.
- Customer data is a **transitive dependency** — its lifetime is the Customer's. A Customer schema change may force a message change.
- Restaurant data is likewise a transitive dependency on the Restaurant schema.

Fat Message — Transitive Dependencies

| Inline someone else's data and you have inherited their release schedule.

five of these eight fields are somebody else's schema

Purchase Order Message



the two red blocks give the purchase order two more reasons to change, and neither of them is a reason of yours

Skinny Message

With a **Skinny Message**, the requestor provides only information unique to the event, and assumes the provider has, or can obtain, the other information.

- The resulting notification message is normally skinny.
- **Example.** *Order Notification → Courier Assignment*: notifies that there is an order, inlining only order-unique data — not reference data like weights or addresses. Courier Assignment must source the missing information from elsewhere.

What it costs you: the provider can no longer act alone. Something else has to supply the rest — which is the next sub-topic.

The Lifetime Rule

| If the data does not share the message's lifetime, put an id in the message, not the data.

The purchase-order message carries `CustomerId` and `RestaurantId` instead of inlined data.

- Data that shares the message's lifetime → **inline it**.
- Data with its own lifetime → **reference it** by id; assume the requestor obtains the data out-of-band.
- These ids must be looked up with other providers (Customer, Restaurant).

Then be pragmatic. Exactly as with a database, you may **denormalise** — inline a common lookup so you do not pay for it on every message.

- Do it for the lookups that actually hurt, not by default.
- The moment you copy someone else's data into your message you own a **stale copy**, and you have taken on their schema.
- The rule is the default; a denormalisation is a decision you should be able to justify.

6.2 Reference Data

How the provider gets what the message did not carry.

Reference Data

The requestor assumes **Provider A** has a local cache of data from **Provider B** that it can use to look up the identifiers in a skinny message.

- Data that leaves Provider B via an API is **Reference Data**: immutable, versioned, stale.
- Immutable and versioned because a copy that can change shape under its holder cannot be cached safely.
- It can be cached locally to Provider A to avoid frequent lookups (cache lookup()).

Get It On Demand — REST/RPC

On a cache miss, Provider A fetches the data from Provider B synchronously.

- Look up missing data in the local reference-data cache (may specify identity *and* version).
- On a miss, request from Provider B and store it in the cache (`request()` / `reply()` / `cache write()`).

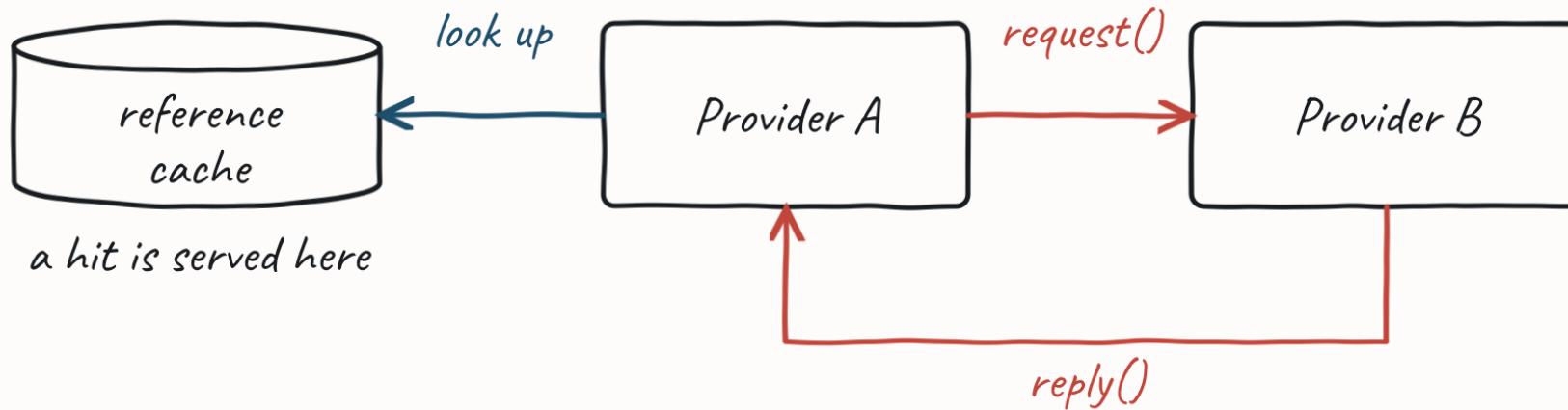
What it costs you

- On a hit: **availability over consistency** — you serve possibly-stale data and stay up.
- On a miss: **consistency over availability** — and A's uptime becomes the uptime of **A and B**. If B fails, A fails.

Get It On Demand – REST/RPC

| A cache miss is a temporal coupling you did not plan for.

on a miss, A is up only while B is up



*on a hit: possibly-stale data, and you stay up
on a miss: the availabilities multiply again*

Content Enricher — Someone Else Does the Lookup

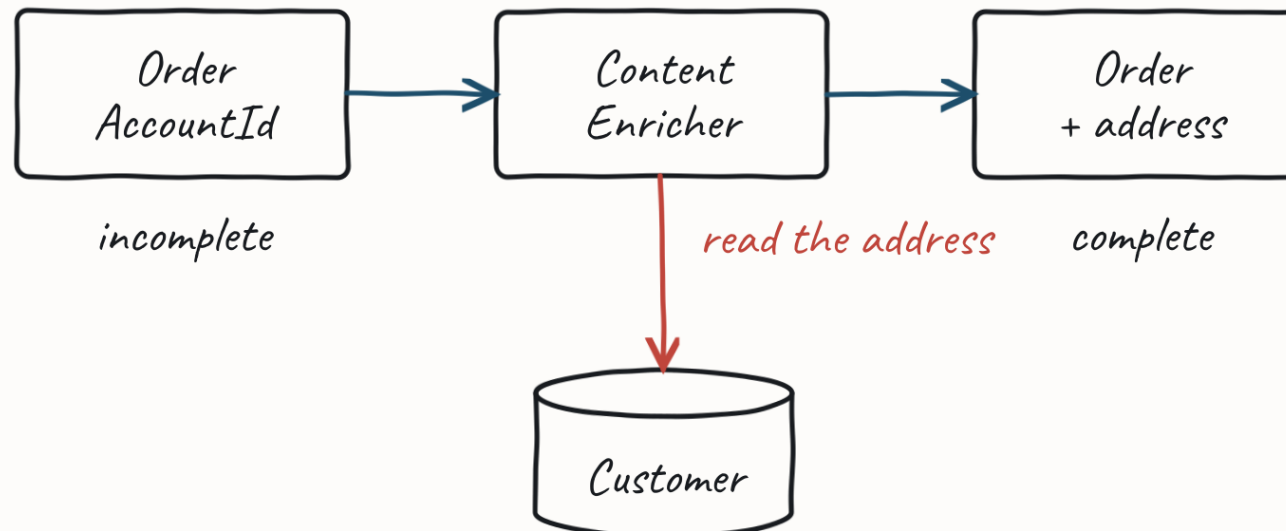
A **Content Enricher** is a filter step that adds required data the publisher did not include: it listens on the channel, fetches what is missing, and republishes the message complete.

- A new order carries an `AccountId`, but shipping needs the customer's address. The enricher adds the address, and the shipping consumer then has what it needs.
- Especially common in microservices, where each service holds only its own data and something has to do the join.

Content Enricher — Someone Else Does the Lookup

| The enricher does not remove the lookup. It moves it — and the availability sum moves with it.

the lookup did not go away -- it moved here



*and now the shipping consumer depends on the enricher too --
the same availability sum, one hop further away*

Get It In Advance — ECST (Event-Carried State Transfer)

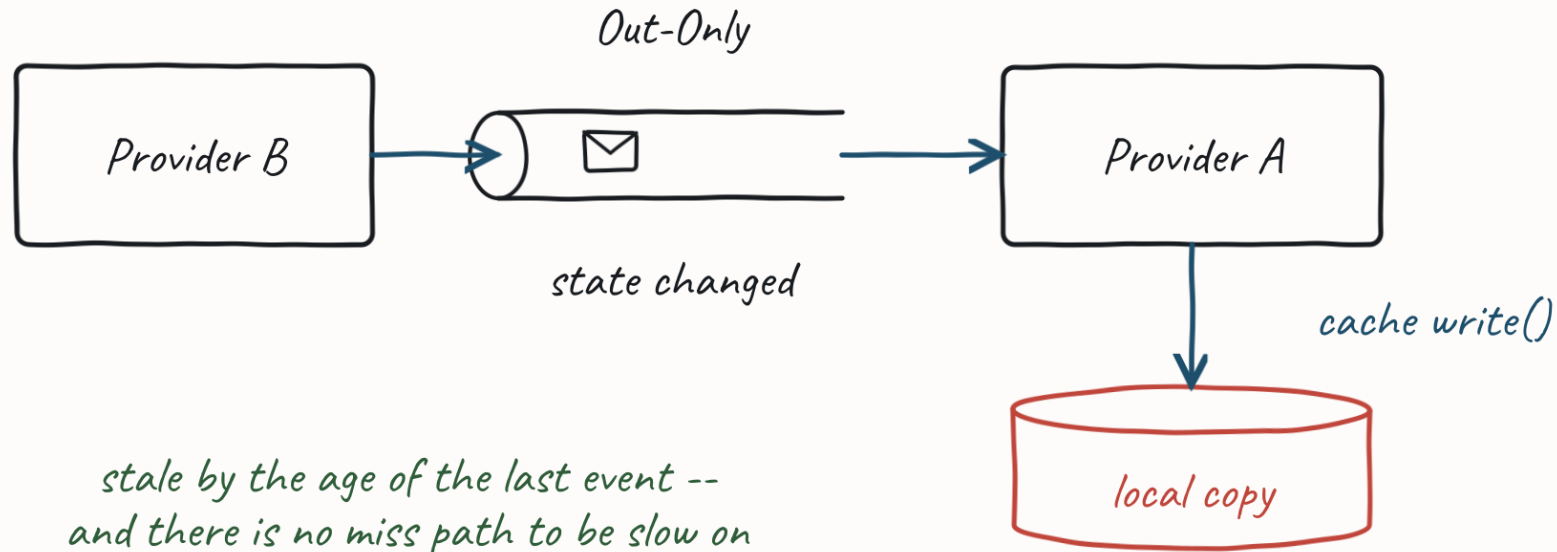
Provider B **publishes** its state changes; Provider A subscribes and keeps a local copy. A never makes a synchronous call to read it.

- The upstream provider raises a **notification** when its own entity state changes (Out-Only / pub-sub).
- The downstream provider subscribes and writes to its local cache (`cache write()`); every request then reads locally.
- There is **no miss path**, so there is no synchronous call in the middle of a message flow.

Get It In Advance — ECST (Event-Carried State Transfer)

| This is the default. Prefer it to the synchronous lookup.

every read is local -- nothing here calls anyone



Why It Holds Up in Practice

- **Latency is not the problem people expect it to be.** Propagation is a broker hop — you are behind by milliseconds to seconds. And this is *reference* data: restaurants, customers, price lists. It changes rarely, and rarely in a way the next message depends on.
- **Versioning is what makes it safe.** Carry **id and version**. Then "I have not seen this yet" is distinguishable from "I have it", and a missing version becomes a **wait** rather than a wrong answer — which the worked example shows.
- **You removed the temporal coupling rather than relocating it.** A stays up when B is down. Day 1 §Coupling's *Must We Both Be Up?* arriving inside message design.

Be Honest About the Trade — and Which Way It Runs

ECST chooses **availability over consistency** on purpose, and by an amount you can measure: you read a copy that is behind by the propagation delay. The synchronous lookup is not the consistent option — it makes the same trade on a cache hit, and then **reverses it on a miss**, when B is down and you are not.

What it actually costs: you are running a replica, so you own a subscription and you must be able to **rebuild** it — replay the stream from the beginning. That is operational work, not a correctness risk.

When to still reach for the lookup: the data genuinely cannot be replicated — too large, too sensitive, or it must be fresh at the instant you read it (an authorisation or a balance check). Then take the coupling knowingly, and put a circuit breaker on it.

Reference Data — Worked Example

Order Fulfilment → Courier Assignment: the request omits the restaurant pickup address; we assume Courier Assignment obtained it from Restaurant Information.

- Look up the restaurant in the local cache by **id and version**.
- If we have the restaurant but **not that version**, apply **backpressure** and retry the order after a delay.

| A missing version is a wait. A missing *value* would have been a wrong answer.

6.3 Event Shape

What shape of event lets the receiver keep a copy — and what guarantees that buys.

Domain or Delta Event

An approach to Pub-Sub where the provider communicates **granular** state changes to its requestors.

- Messages are usually named as a past participle after the causing command (e.g. `BasketItemRemoved`, `BasketItemAdded`).
- Cannot use a **Datatype Channel** — domain events for the observable must be *ordered on the same channel* and have different schemas. This burdens the requestor to multiplex handling of the events.

Summary or Snapshot Event

An approach to Pub-Sub where the provider communicates a **summary** of state changes.

- The message is **versioned** and contains metadata describing the cause(s) of changes; usually named after the observable (e.g. BasketChanged).
- *Can* use a Datatype Channel — there is just one schema for a snapshot of the observable.

Why ECST Needs Snapshots

You cannot replicate someone else's state from deltas unless you receive **every one of them, in order**.

- With **Domain/Delta events**, a missed or reordered message leaves the replica permanently wrong — and nothing in the stream tells you. You must apply them all, you can only use blocking retry, and you cannot shed load.
- With **Summary/Snapshot events**, each message is complete in itself. A missed one is repaired by the next one to arrive.

Choose the delta and you have chosen strict ordering. Choose the snapshot and you have bought it back.

If Later, Stream

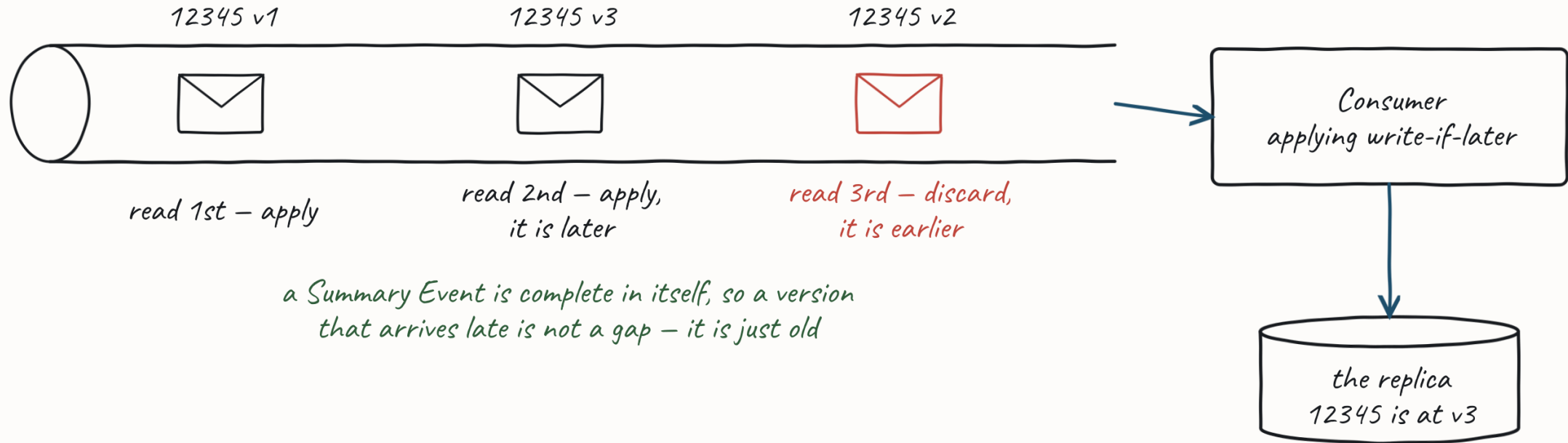
If Later lets us use a versioned Summary Event to ignore ordering errors.

- Consumer reads v1 of 12345 and handles it; reads v3 and applies it (later than v1); reads v2 and **discards** it (earlier than the already-applied v3).
- Even on a stream, non-blocking retry or guaranteed delivery via an outbox can produce out-of-order messages; If-Later also lets us shed load.
- We *cannot* use If-Later with a Domain Event — those must all be applied. With Domain Events we can only use a blocking retry and cannot shed load.

If Later, Stream

the late message is discarded, not repaired – v3 already says everything v2 says

the consumer reads left to right

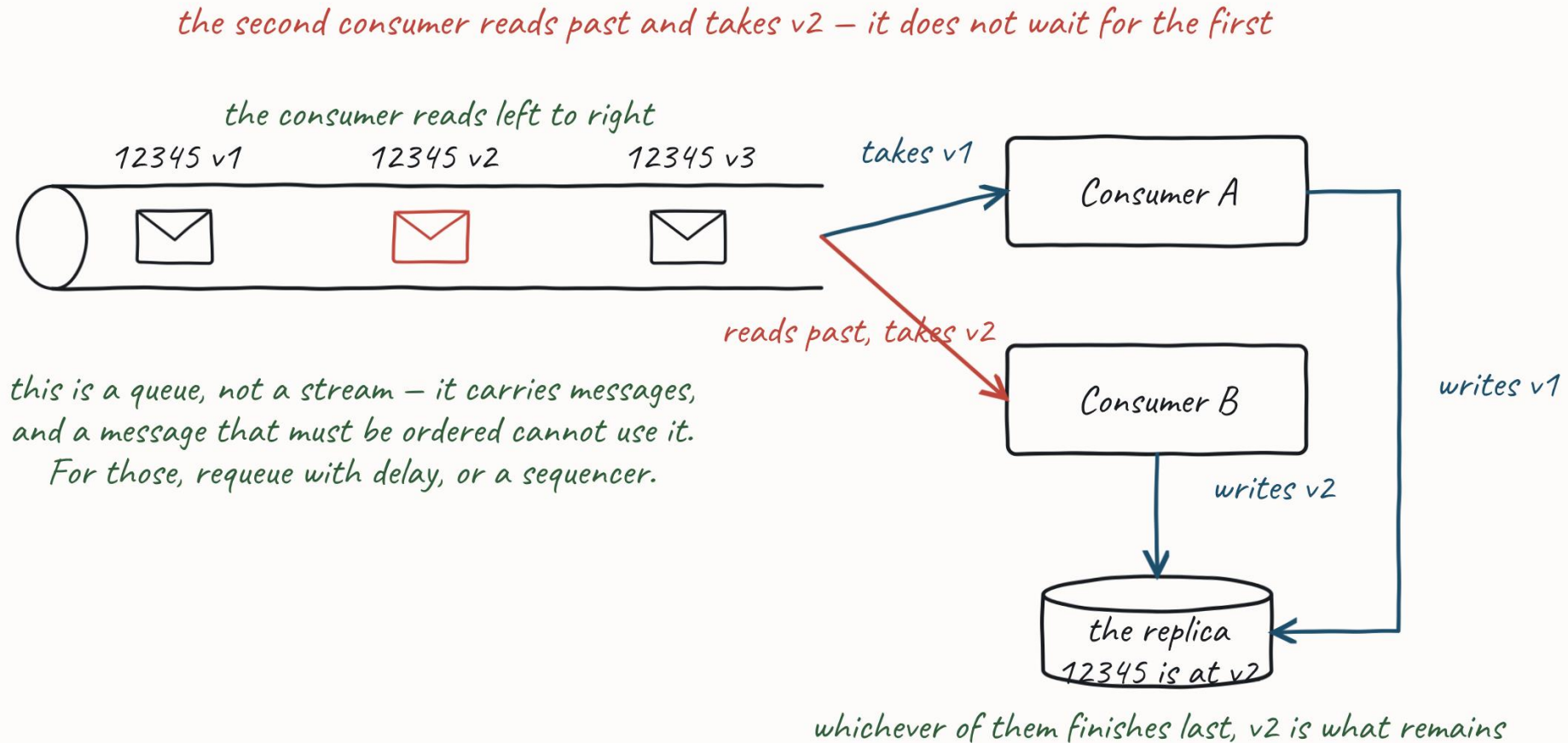


If Later, Queue

If-Later also lets messages be processed out-of-order with a queue and competing consumers.

- One consumer expects/writes v1 while another writes v2; **read-past** lets them proceed.
- A queue normally processes messages (not events), so this applies only where we use a queue.
- If a message *must* be ordered (e.g. a series of commands), use `requeue-with-delay` or a sequencer to re-order.

If Later, Queue



Public and Private Providers

An approach to Pub-Sub where a **public** provider communicates with collaborators in other domains via a **Summary Event** aggregated from the domain events of **private** providers.

- Private providers raise granular event() s.
- The public provider republishes a versioned summary() with metadata describing the cause(s) of changes.

Closing

Where to go deeper, and what to ask.

Further Reading — Data on the Outside

Data on the Outside versus Data on the Inside — Pat Helland, CIDR 2005. Free online.

The paper behind this morning: data that crosses a process boundary is a **copy**, immutable and already stale, and that is why it travels as a message rather than a call.

Further Reading — Building Microservices

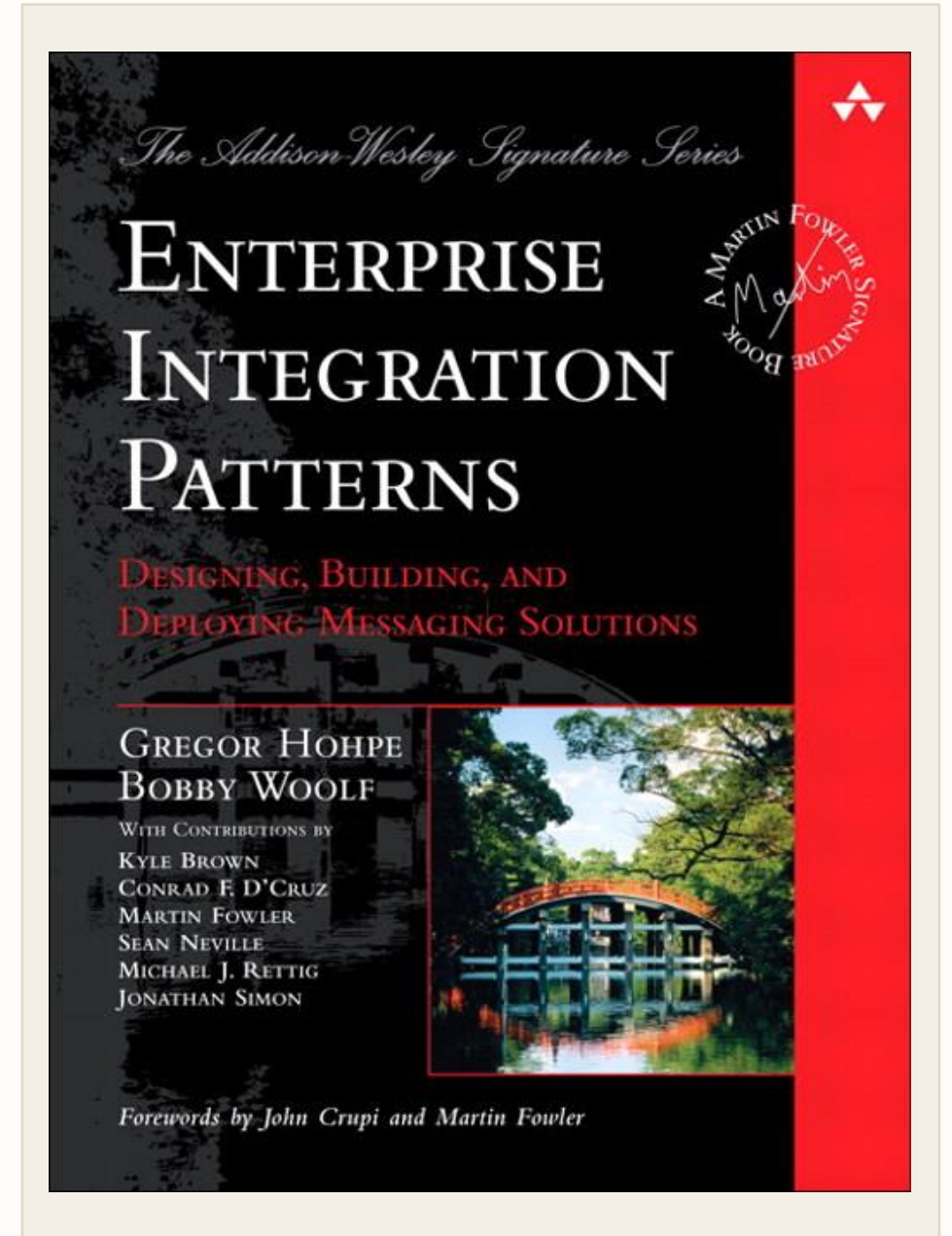
Building Microservices — Sam Newman (O'Reilly, 2nd edition). samnewman.io

For §Coupling and §Integration Styles — the same four styles on the same grid, argued at book length, and the boundary as an **organisational** one as much as a technical one.

Further Reading — Enterprise Integration Patterns

Enterprise Integration Patterns — Gregor Hohpe & Bobby Woolf. enterpriseintegrationpatterns.com

The catalogue §Messaging Patterns follows. Every pattern named today is written up there in full, with the trade-offs the slides had to compress.



Further Reading — Designing Data-Intensive Applications

Designing Data-Intensive Applications — Martin Kleppmann (O'Reilly). dataintensive.net

For §4.5 Queues and Streams and §6.3 Event Shape — ordering, idempotency, replication, and why a log and a queue are not the same thing.

CLOSING

Q&A

Questions and discussion.